# Group-04: IoT for Citizen Engagement in Smart Cities 2025

# Project Overview

## General Description

According to the assignment received as part of the course, the project is aimed at promoting citizen engagement in the development of smart cities through the use of Internet of Things (IoT) technologies. In its initial description, it was intended to be developed in cooperation with Stadtwerke München (SWM), which determined the focus on the city of Munich. The stated objective was to develop a solution capable of collecting, analyzing, and visualizing real-time data, contributing to the enhancement of informational transparency and public awareness.

The defined task was too broad and required clarification and further specification, which forced us to look for a particular niche within the urban environment that would be both relevant to residents and feasible to implement using IoT technologies. Based on personal experiences collected while living in Munich and the understanding of its urban context, emphasis was laid on street lighting and nighttime safety. Despite Munich's reputation as one of the safest cities in Germany, there are still areas where safety could be improved. Inadequate street lighting, especially at night and in adverse weather conditions (e.g. autumn fog), reduces the subjective feeling of safety and increases risks for vulnerable groups. However, citizens frequently lack the capacity to report problem areas and participate in planning their improvement.

The developed system makes use of a  mobile Internet of Things device with sensors that can be mounted on a variety of vehicles, such as e-scooters and buses. These sensors transmit data in real-time, record weather and light levels, and identify infrastructure problems or areas with low lighting. Users can view the data collected rom the study area and leave geotagged comments on insufficient lighting in an interactive web application.

The web application solution developed is aimed at a wide range of users and can be useful for both residents and visitors to the city, as well as municipal services responsible for the operation and planning of street lighting. The full system could be scaled to cover bigger areas even up to  entire cities.

## Objectives

Although the feeling of safety in an urban environment is not determined solely by the level of lighting or weather conditions, this project can contribute to improving pedestrian safety, especially for women, children, and other vulnerable groups, by identifying poorly lit areas in a timely manner and helping to improve urban infrastructure. It also provides conditions for more active citizen participation in shaping the urban environment through a transparent system of monitoring and feedback.

The planned development involves the creation of a tool that integrates data on weather conditions and lighting levels into a unified system for data collection and analysis.

The possibility of integrating the project into the SWM infrastructure was considered, including the implementation of user profiles with gamification elements, such as awarding points to active participants, which could then be exchanged for benefits when using public transport.

Since the project is carried out for educational purposes, the main goal is to create a functional proof-of-concept solution that confirms the technical feasibility of using mobile IoT sensors to collect and transmit data on lighting levels and related parameters, with subsequent visualization. All key functions are planned to be implemented and tested. Basing on this main goal the following specific objectives were outlined:

1. To create an IoT setup that senses, transmits and stores lighting and environmental data
2. To create appropriate dashboards that can be used to visualize and hence analyse the collected data.
3. To develop a web application that allows users to interact with collected data, give feedback and information which can be actionable by policy makers. Give a voice to the citizens in other words.
4. To analyse the data, map lighting levels in a visual interactive map and investigate if there is possible interaction between visible light and environmental factors.

The success criteria will be the stable operation of the hardware, the correct transmission of sensor data, its display and updating on the web platform, as well as the overall alignment of the interface with the stated concept. All key elements of the idea (data collection, transmission, visualisation, and feedback) must be implemented at least in the basic version. A successful demonstration should confirm that the proposed concept is technically feasible and can serve as a basis for further development in real-world conditions.

## Site selection

For our study, we selected an area near the Moosfeld subway station. It includes low-rise urban housing, small zones with commercial premises on the ground floor, as well as streets located along agricultural fields. In other words, this area offers a high degree of variability in types of urban environments, and also represents an interesting location for the goals of our lighting study.


Selected anchor points illustrating perceived safety conditions across the site

Some areas (Karotschstraße, Emplstraße, and Am Mitterfeld towards Schmuckerweg) within the residential development are not perceived as unsafe. This can be explained by high population density, good sound permeability, and active evening use.

Route segments running along the agricultural field (Truchthari-Anger, Straßl ins Holz), despite the presence of residential buildings on one side of the street, are subjectively perceived as less safe. This perception is caused by several factors at once: low lighting levels, weak street activity during late hours, visual isolation (some houses are uninhabited or poorly lit), as well as acoustic silence, which eliminates the possibility of attracting attention in an emergency situation.

In such areas, the feeling of "public surveillance" is absent, which, according to the concept of Jane Jacobs, reduces pedestrians' subjective sense of safety.

## Literature Review

Since, in the framework of our project, we decided to focus on the topic of urban safety, in our literature review, we also searched for scientific articles in this field. The authors of the article "*Perspectives on urban lighting: Pedestrian visual comfort and safety preferences*" aimed to understand how pedestrians perceive different urban lighting scenarios in real nighttime conditions. Specifically, they tried to determine clear preferences for lamp post height and the uniformity of their distribution — that is, the distance between the posts. They studied how these parameters influence the perception of comfort, safety, and spatial orientation. They conducted a lab experiment among participants familiar with the location — the square in front of the central train station in the city of Ljubljana. The lighting scenarios were simulated using Dialux (4 scenes with high and low lamp posts and high and low uniformity in their distribution). Participants evaluated these scenes based on four criteria: visual comfort, confidence, ease of orientation, and distance estimation. The results are presented in the figure from the article. As a result of the study, it was found that lighting uniformity has a significant impact on all four subjective metrics, while the height of the posts by itself did not show statistically significant differences. However, it is important to understand that the height of the posts increases the level of light pollution.

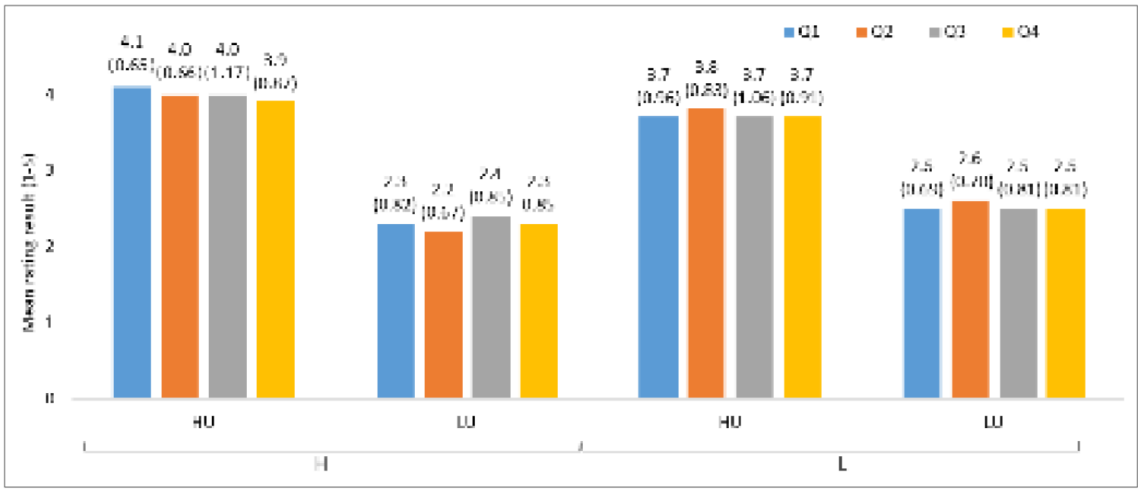



Figure 1. Mean rating results and variations (the number in parentheses) for all lighting scenes, categorized as higher uniform (HU), lower uniform (LU), high lamp-post (H), and low lamp-post (L). The ratings are based on four questions: visual comfort (Q1), confidence (Q2), ease of orientation (Q3), and ease of perceiving distances (Q4). [Number of article in references ]

Nevertheless, although lighting uniformity is an important factor in ensuring a sense of safety in urban environments, it is also important to consider that a high level of illumination at night may result in high electricity consumption. In this sense, an interesting solution in the field of SMART lighting was presented by the authors of another study "*Distributed Architectures for Intensive Urban Computing: A Case Study on Smart Lighting for Sustainable Cities*". They propose to use video analysis to turn on streetlights only when people are detected on the street, and to adjust the lighting level in three steps — 0%, 20%, 100% — depending on human activity. This allows for energy savings and a reduction in light pollution. To implement this idea, they propose to use their own unique distributed cloud architecture in order to more effectively deal with the huge volume of video data streams. For this, they integrate a local mini-server (cloudlet) into the standard cloud structure. Thus, things — in this case, cameras — capture video, process the images according to lighting conditions (evening/night), and identify areas in the image where people are potentially located. The local cloudlet nodes perform almost the full range of the remaining necessary data processing steps, namely: analysis of shape, texture, and other important parameters of the object for the system to work, so that it is not necessary to send the full image to the server. A compact descriptor of the human is created, followed by a classification process that determines the presence of a person in the frame. These local nodes also often perform behavioral analysis — that is, trajectory analysis — and make a decision about whether to turn on the light along the route of the person's movement. These nodes also often activate the streetlight — that is, send the command to turn on the light at a specific pole or group of lights. However, the last two steps are also often performed by the cloud server if the two lower levels cannot handle the load. Interestingly, in the framework of their study, different types of cameras were tested, and some of them were not able to perform all three primary data processing steps. Nevertheless, their experiments show that the system can also work with devices of different performance levels. This system demonstrates high accuracy in pedestrian detection compared to infrared sensors and also scalability — this architecture allows for easy addition of new nodes to the system.

Interestingly, the authors of the study mention that this approach can be adapted to different tasks and applied using mobile devices, allowing scalability of our approach, as well as opening opportunities for further development of our idea and its integration into the existing urban infrastructure, where cameras are often already used for monitoring car traffic in the city.

While making a literature overview for our project, we also discovered that various weather conditions can, in fact, affect how safe pedestrians feel when walking in public spaces. A large field study in Israel (30,000+ observations across three cities) found that moderate temperatures (~15–16 °C) and higher humidity levels correlated with a greater “feeling of safety” (FoS) among pedestrians, whereas very hot or very dry conditions reduced perceived safety (Saad et al. 2024). In fact, when nighttime temperature rose from 25 °C to 30 °C, people felt less safe, and the study noted that roughly 20 extra lux of street lighting would be needed to offset this drop in safety perception. Conversely, on cooler or more humid nights, slightly dimmer lighting could achieve the same comfort level, suggesting weather-sensitive lighting strategies for cities. Overall, people in this study felt safest on comfortably warm, “wet” nights (high humidity), and felt less secure during uncomfortably hot or dry conditions (Saad et al. 2024).

Other studies support that inclement weather can heighten pedestrians’ anxiety and risk perception. For example, Zhu et al. (2025) used immersive virtual reality to examine pedestrian road-crossing safety perception under different weather conditions. They found that rainy or foggy conditions led to higher perceived risk (lower perceived safety), especially during dusk when visibility was poor (Zhu et al. 2025). In other words, pedestrians felt more in danger crossing the street in rain/fog (particularly at low-light times) than in clear daytime conditions. This aligns with intuitive factors: heavy rain, fog, or snow reduce visibility and can make streets feel more hazardous, both by increasing the chance of traffic accidents and by making it harder for pedestrians to see their surroundings.

However, since the main topic of our project work this semester was dedicated to citizen engagement in the context of IoT implementation, we also decided to shift the focus from the development of the IoT technology and infrastructure itself to exploring the potential for real citizen participation in working with urban data and designing urban services. The authors of the article "*Opening up Smart Cities: Citizen-Centric Challenges and Opportunities from GIScience*" highlight three main groups of citizen-centric challenges: empowering citizens, citizen-centric services, and analytical methods and tools.

Within the category of empowering citizens, several important aspects are mentioned. Deep participation invites us to see citizens not just as “human sensors collecting data” but as real collaborators. The second aspect, noted by the authors — data literate citizenry — is that citizens should not only be consumers of digital services, but also understand how data works, recognize the importance of proactive participation in city governance, the importance of caring for the environment, and be motivated to improve the quality of life. To achieve this, the authors call for the development of digital inclusion, meaning access to technology for

everyone regardless of age, education, or social status. This is only possible through centralized efforts to raise awareness among citizens by organizing educational initiatives from the side of city governments.

In the subsection Analytical methods and tools, the authors primarily propose combining quantitative and qualitative data through social media, crowdfunding tools, or, for example, citizen science activities. They also call for the development of open standards in order to ensure interoperability between various applications and services within the IoT system.

In the section on citizen-centric services, the authors note that there is a lack of individualized digital services that could adaptively respond to the specific needs of an individual and allow them to filter the information they provide about themselves in all these open IoT services. The authors also highlight that it is very important to change the approach to citizen interaction and to develop such interfaces that show the right information at the right time and encourage people to act. This approach is especially relevant, for example, in solving environmental problems, where active citizen participation plays a key role.

For each of these directions, there are already certain tools in the field of GIScience, showing significant technological development in this area. However, the authors also highlight directions for future research and development, which are presented in the Figure below.

| Research Challenges | GEO-C upcoming Features Beyond the State-of-the-Art |
|---|---|
| Deep participation (**C1**) | • Identifying and understanding the main motivating factors that characterise online citizen participation<br>• Explore the concept of virtual meeting geo-spaces to bridge the gaps between VGI and PPGIS<br>• Public displays as integrators in open and smart cities; rethink the traditional concept of map as big data analysis, cartography, and visual art |
| Data literate citizenry (**C2**) | • Educational tools for children to become citizen scientists<br>• Active, customized open data maps that facilitate its full understanding to distinct groups of citizens |
| Pairing quantitative and qualitative data **(C3)** | • Methods to integrate spatiotemporal quantitative measurements and predictions with qualitative assessments about an individual location<br>• Methods to downscale coarse climatic data at city level<br>• Predictive analytics for improved citizen mobility based on social networks and citizen's digital footprints<br>• Analysis of spatiotemporal interactions of crime data to predict crime hotspots in cities using data provided by the Web 2.0 |
| Adoption of open standards **(C4)** | • Framework for creating and deploying standards-based participatory sensing applications<br>• Standards-driven data hubs for accessing and exposing real-time data streams |
| Personal services **(C5)** | • Methods for proximity-based opportunistic information sharing and privacy protection<br>• Determining social roles and relationships between nearby devices and/or services |
| Persuasive interfaces **(C6)** | • Geospatial technology and visual interfaces for green behavior and/or living<br>• Social implications of geospatial technology and location-aware interfaces for behavior changes |

Figure 2. Example research directions at the intersection between GIScience and smart cities. [Number of article in references ]

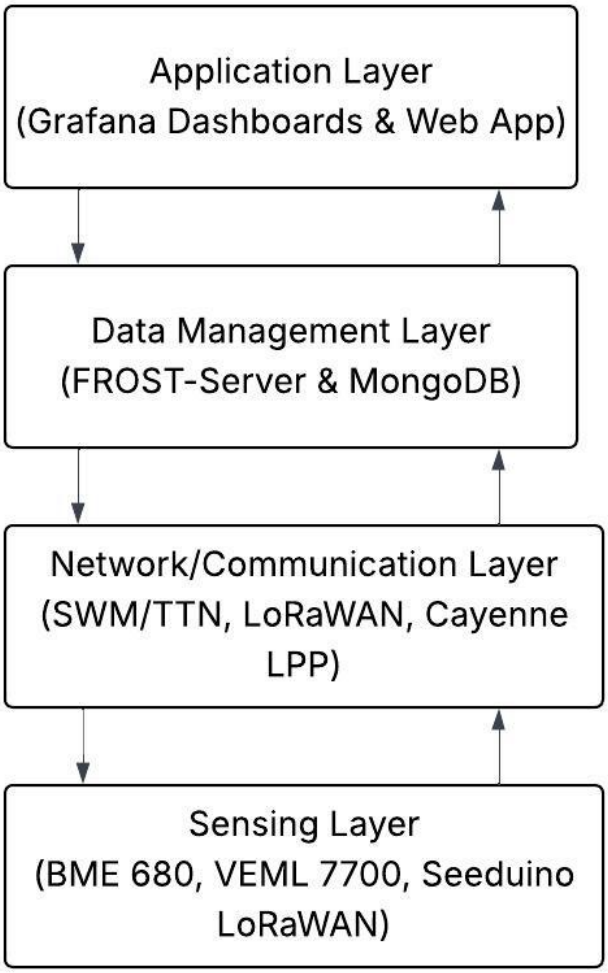
# System Design & Methodology

The system was designed to monitor street lighting conditions using a light sensor and an environmental conditions sensor, process the data using an IoT platform and visualize it via Grafana dashboards and a standalone web application, that in addition to visualisation, enables users to provide feedbacks, reports and data that can be useful to the city planners and city managers.

## Overall Architecture

The system followed a layered IoT architecture which enabled scalability and ease of maintenance. The high-level architecture was made up of four layers, namely:

1. **Sensing Layer:** This consists of the two sensors, BME 680 environmental sensors, which measured the temperature, humidity, and pressure, and the VEML 7700 Light sensor, which measured the illumination at different locations during the data collection process. Seeeduino LoRaWAN microcontroller was used, data was encoded into Cayenne LPP (Low Power Payload) format which is offers efficient and compact transmission.
2. **Network/Communication Layer:** The protocol used was LoRaWAN which has low-power and long-range communication capability. The format of payloads transmitted was the standardized Cayenne LPP format for consistency and efficiency purposes. Cayenne LPP is energy efficient for battery-operated things and resilient against connectivity. While there are a lot of advantages attached to this format, there were some shortcomings that presented challenges when processing the collected data, which will be discussed in the Discussion section. Data was transmitted through The Things Networks(TTN) and SWM. TTN was majorly used when working inside the city and SWM was used in the city outskirts because it had better connection there.
3. **Data Management Layer**: Here the FROST-Server which acts as the middleware implementing the OCG SensorThings API standard hence handling sensor entity management, was used. FROST-Server is full open source and offers transparency and longevity to users. All the Things, Sensors, Datastreams, Observations and Locations were managed through the FORST-Server. The Database used for persistent data was MongoDB which allows time-series data storage, scalability and support for geospatial queries. Incase of connectivity disruptions, the nodes buffered data until transmission restoration.
4. **Application Layer**: Grafana Dashboards and a standalone Web application were used for visualization and user interactive applications. Grafana dashboards were set up directly from the persistent stream of data in the MongoDB, and they provided analytical views for the technical team. The web application was designed for public use, and data was retrieved manually from the MongoDB and visualised through Chart.js. Additionally it featured, interactive maps, safety reports among other components.



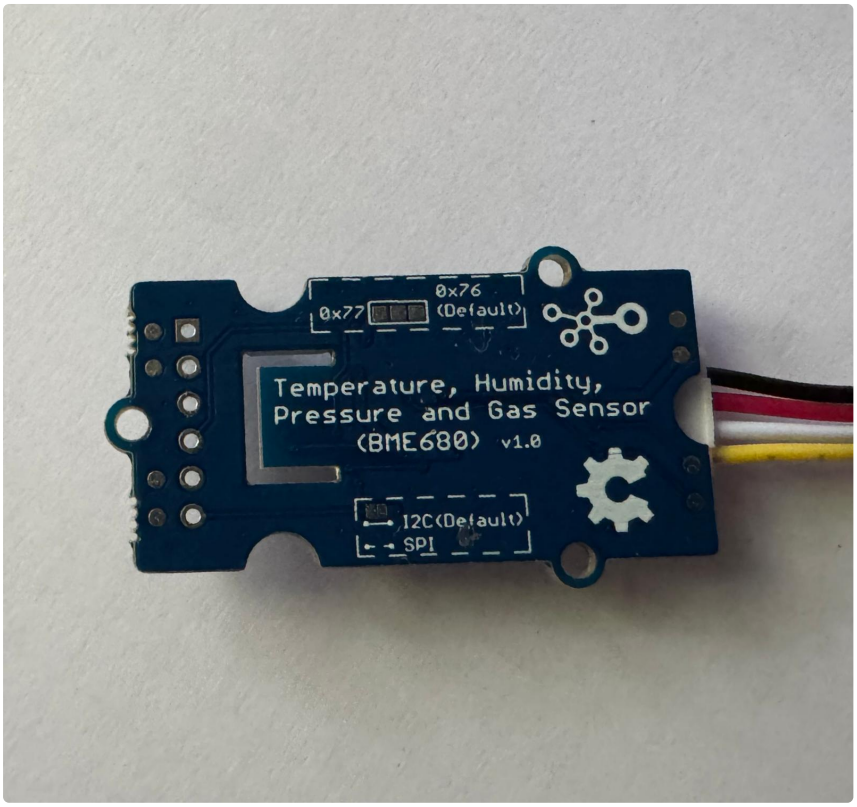

Overall System Architecture

## Sensors Used

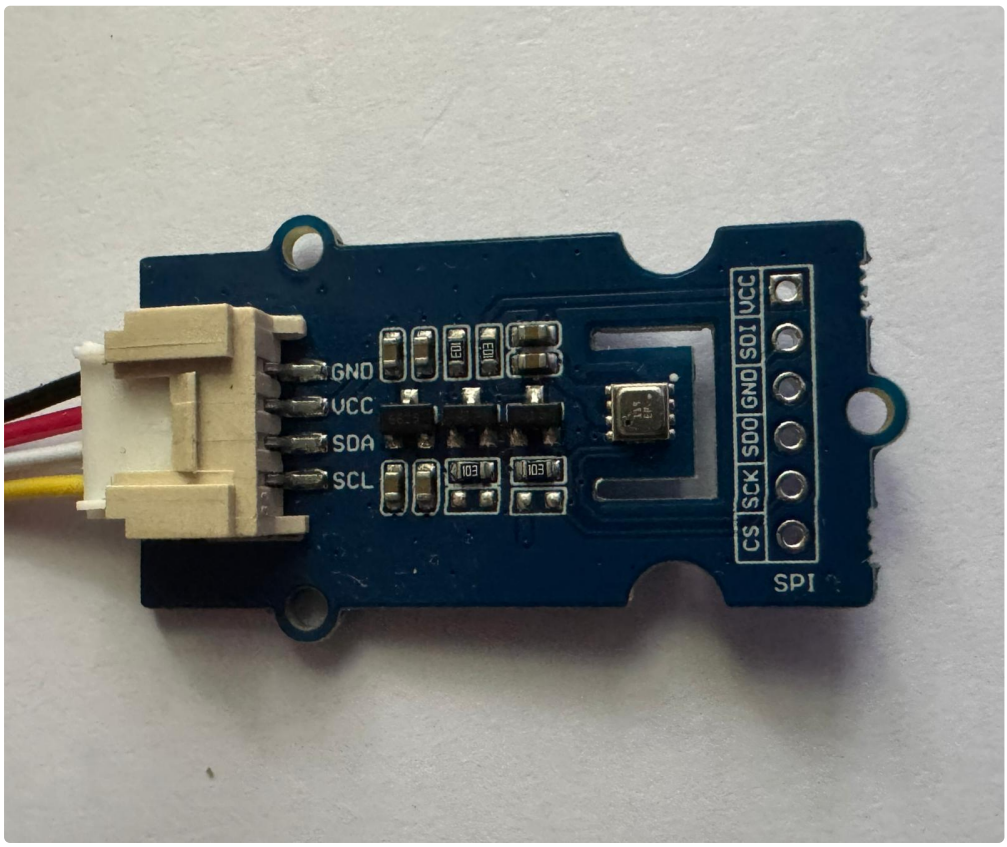
The sensor selection prioritised high-accuracy and low-power sensors suitable for outdoor deployment t monitor street lighting and environmental conditions at night. The BME 680 sensor was chosen for environmental measurements, and the VEML 7700 sensor was chosen for light intensity measurements. Both were integrated with Seeduino LoRaWAN microcontroller.

- **Environmental Sensor: Grove-Temperature Humidity Pressure Gas Sensor(BME 680)**
  - Description: The BME 680 is a 4-in-1 compact , digital sensor developed by Seeed Studio based on Bosch's Sensor chip. It measures temperature, humidity, barometric pressure and volatile organic compounds (VOCs) for air quality assessment, with a single device making it very useful for environmental monitoring.
  - Features:

    - 4-in-1  for multiple measurement
    - Low power consumption
    - Wide measurement range
    - Optional: Individual humidity, pressure, gas , temperature can be independently enabled or disabled.
- Specifications:
    - Working Voltage: 3.3V/5V
    - Operating Range:
        - Temperature :  -40~+85℃
        - Humidity: 0-100%
        - Pressure: 300-1100hPa
    - Digital Interface: I2C(up to 3.4MHZ)/ SPI (3 and 4 wire, up to 10MHZ)
    - I2C address: 0x76(default) 0x77(optional)
    - platforms supported: Arduino, Raspberry Pi, ESP IDF




BME Sensor

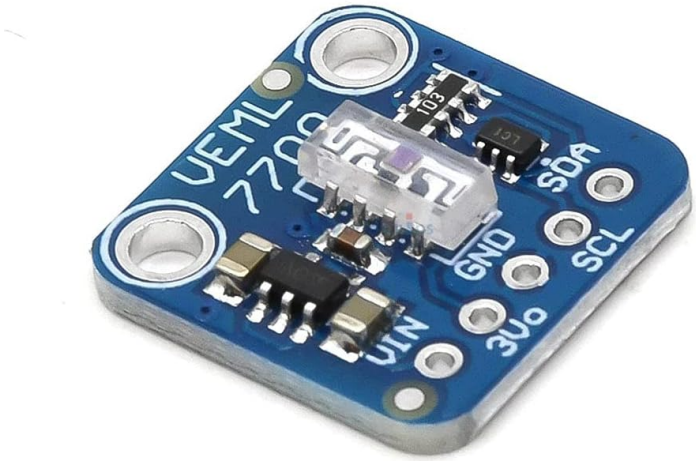



- Justification:
  - The BME 680 4-in-1 feature enables the monitoring of comprehensive environmental data. For instance additional metrics like pressure and VOCs are provided in BME 680 unlike the DHT22 and other sensors. This enables a holistic assessment of environmental impacts on lighting perception.
  - This sensor consumes ultra-low power , this allows for extended battery life which is crucial for scalable street deployments.
  - The BME 680 has a good software support provided by the BSEC library. The BSEC library provides pre-calibrated IAQ calculations and simplifies data processing.
  - BME 680 is readily compatible with Seeduino LoRaWAN which was the micro-controller used in this project. The Grove BME 680 is specifically designed for Seeed Studio's ecosystem under which the Seeeduino LoRaWAN falls. This eliminated complex wiring and reduced assembly errors.
  - Readily available Seed Studio's Grove ~~BME 680 Arduino library providing pre~~ - built functions for reading temperature, pressure, humidity, and VOCs ensured seamless integration and minimized firmware development time.

- **Light Sensor: Adafruit VEML 7700 Ambient Light Sensor.**
  - Description: The VEML 7700 sensor is produced by Vishay Semiconductors. It is a high-accuracy ambient light sensor that mimics the human eye's response to light . It is used for precise illuminance measurements in lux and it is ideal for outdoor applications.
  - Features:

- Integrated modules: Ambient Light sensor(ALS)
- Low shut down current consumption: 0.5 µAFiltron technology which mimics human eye response

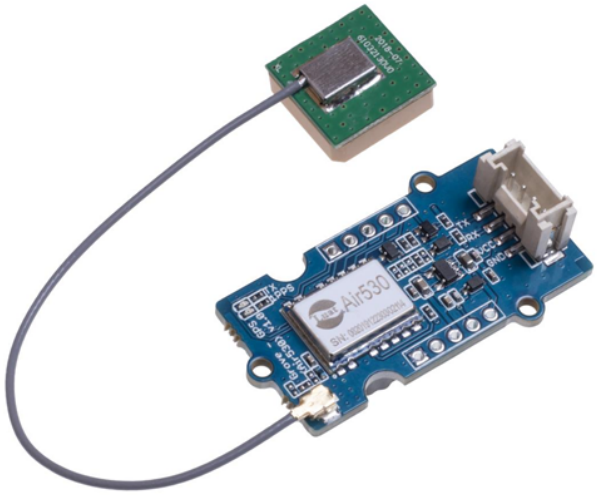
- Specifications:
  - Measurement Range - 0-120,000 lux, with  adaptive gain for low and high light conditions
  - Supply Voltage Range :2.5V to 3.6V
  - Communication: via I2C interface.
  - Supports 100 Hz to 120 Hz flicker noise rejection.




VEML 7700 Sensor


- Justification:
  - High sensitivity and precision. The VEML 7700 has a precision of +/-0.1 lux and 10% accuracy that surpasses many digital sensors such as the TSL2561 previously used in the project, which struggled to accurately measure low light levels. The VEML 7700's adaptive gain dynamically adjusts to ambient conditions, ensuring consistent performance which was useful  throughout the data collection process.

        - The human-centric Filtron technology for sensing light intensity offered by this sensor aligned well with our objective was geared toward human eye lighting perception in the streets at night.
        - The VEML 7700 I2C interface is compatible with Seeduino LoRaWAN though after extra wiring because it is t in the Grove ecosystem.
        - The wide dynamic range offered by VEML 7700 covers all urban lighting scenarios from dark alleys to bright streets which was required in this project.
- GPS Module: Grove-GPS (Air530/ Air530Z)
    - Description: The Grove GPS Air 530 is a high-performance module from Seeed Studio with Global Navigation Satellite System(GNSS). It is designed for integration with platforms like Arduino and Raspberry Pi via the Grove Connector System. It supports multiple satellite systems which makes it ideal for IoT projects like this one that required precise location data.
    - Features:
        - It has highly integrated Multi-mode satellite positioning and navigation
        - It is tiny in volume and has low power consumption
        - The size is compact which makes it easy for deployment
        - It is cost effective
    - Specifications:
        - Supply voltage: 3.3V/5V
        - Working current: up to 60mA
        - Time of warm start: 4s
        - Time of cold boot: 30s
        - Interface: UART(TXD/RXD at 2.8V TTL), 1PPS, Grove connector.




GPS Module (Air 530)

- Justification:
  - GPS- Air530 has Multi-GNSS support which helps ensure consistent positioning even when one satellite is blocked, this makes it useful in the dense urban setting that the project  was carried out on.
  - This module has a low power consumption rating which was ideal for our battery powered IoT Thing used in the project.
  - Since it is in the Grove ecosystem it allowed for easy integration without the need for soldering or extra wiring.
  - The wide UART baud rate offered by this module made the integration with Seeeduino LoRaWAN microcontroller seamless.
  - The LED Indicators offered real-time status feedback (TX RX activity and precise 1PPS timing), simplifying debugging.

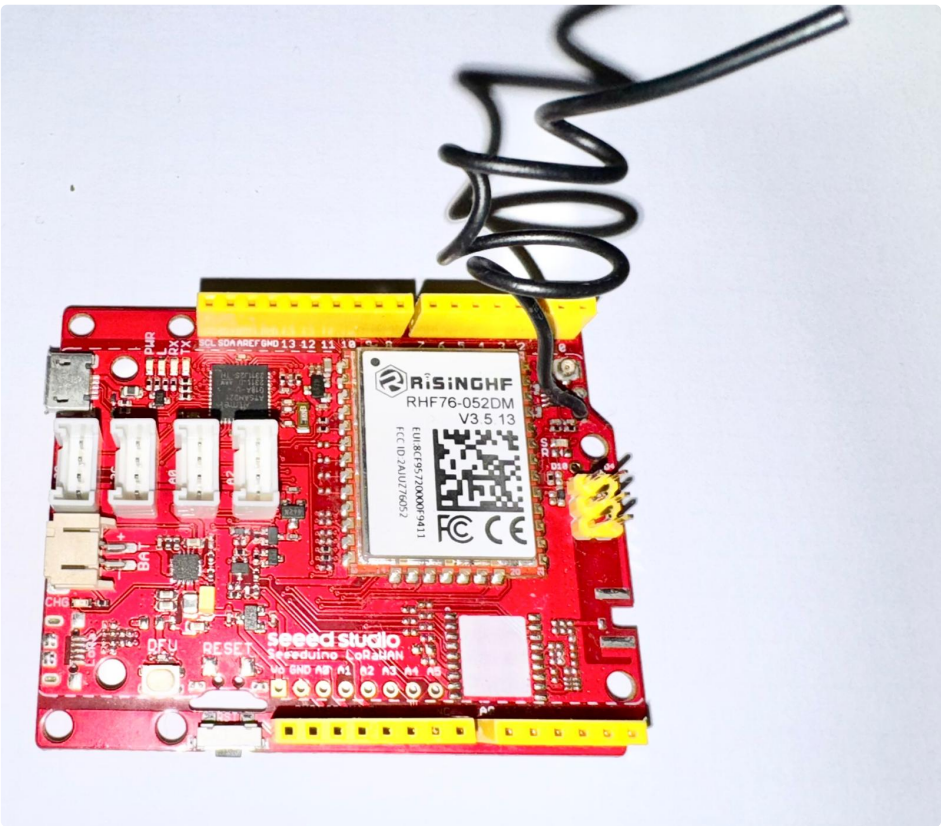
## IoT Platform & Communication

The IoT platform and communication channels were responsible for transmitting sensor data from deployed thing to the backend infrastructure. The components used here leveraged a combination of hardware and protocols optimised for low-energy consumption, wide area coverage and interoperability.

## Microcontroller

The microcontroller used in this project was the **Seeeduino LoRaWAN**, which offers integration of LoRaWAN radio and is compatible with Arduino-based sensor libraries used. This hardware formed the bridge between the sensing layer and theNetwork Layer.

- **Description**: Seeeduino LoRaWAN is an Arduino-compatible development board with LoRaWan protocol embedded, designed for Internet of Things(IoT) applications especially those that require long-range, low-power wireless communication. It is on the communication module RHF76-052AM, which makes itcompatible with LoRaWAN Class A/C and supports a variety of communication frequencies.
- **Features**:
  - Support LoRaWAN protocol, Class  A/C
  - Low Power Consumption: minimum current 2mA on 3.7V lipo battery
  - Battery Management: It is embedded with lithium battery management chip and status led indicator
  - Ultra long range communication
  - Ultra low power consumption
  - Grove Ecosystem Support: It has 4 on-board Grove connectors
  - It is Arduino compatible.
  - Additional peripherals: DFU and Reset buttons, wire/uFL antenna options, Micro USB for power and programming, ICSP pins,  PWM pins among others.
- **Specifications**:
  - Microcontroller: ATSAMD21G18, 32-Bit ARM Cortex M0+
  - Operating voltage: 3.3V,
  - Analog Input Pins: 6,12-bit ADC channels
  - Analog Output Pins: 1,10-bit DAC
  - Digital I/O Pins: 20
  - UART: 2
  - Flash Memory: 256 KB
  - SRAM: 32KB
  - Clock speed: 48MHz
  - DC Current per I/O Pin: 7mA



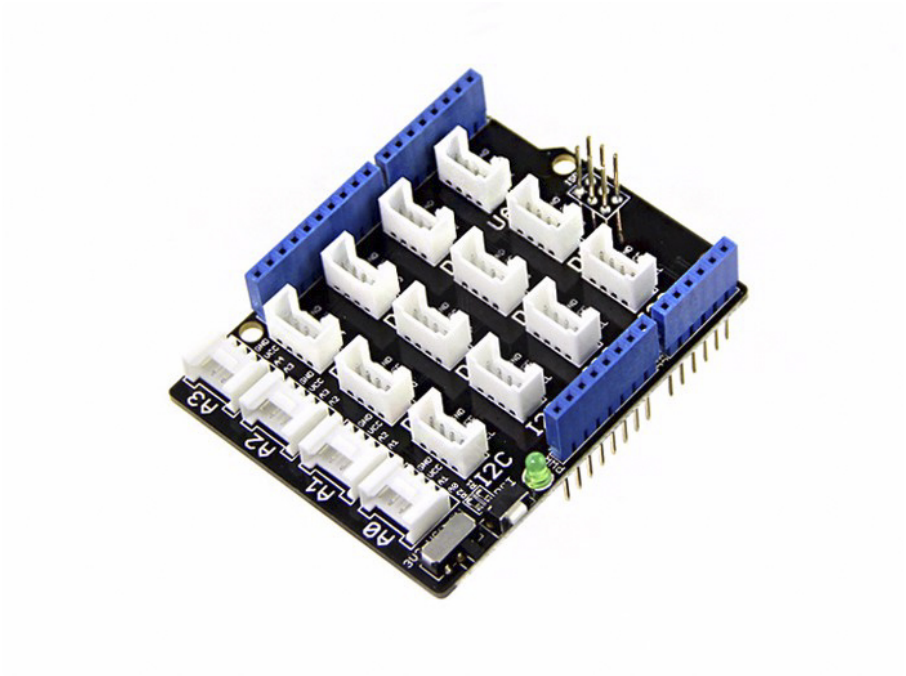

Seeeduino LoRaWAN Microcontroller

- **Justification**:
  - Seeeduino LoRaWAN has long range and low power capabilities, which made it ideal for this project locations which  required communication over kilometers with minimal energy use.
  - This MC is compatible with Arduino which made it easy to integrate and prototype since the pre-built LoRaWAN library was used. Due to the Grove connectors it allowed quick and solder-free connection of the GPS and BME680 modules.
  - The built-in LiPo management and USB charging allowed for an easy use of Lithium battery and a back up USB power bank during data collection.
  - It is cost effective compared to other micro controllers.
  - It offers low-power optimizations which allow for energy efficiency.

## Seeeduino LoRaWAN Expansion: Base Shield V2

- **Description**: The Base Shield V2 board was used to expand Seeeduino LoRaWAN. It simplifies the prototyping with Grove modules. It replaces the cluttering from breadboards and jumper wires with a streamlined interface that enables the user t plug on modules directly.
- Features:
  - It has 16 integrated Grove connectors, including I2C, UART, Digital and Analog connectors

    - It has selectable Power Voltage that allows one to toggle switch between 5V and 3.3 V
    - It has a reset button
    - Base Shield V2 has an LED Power Indicator
    - It has ICSP pins and P1/P2 pads
- Specifications:
    - Operating Voltage: 3.3 V or 5V
    - I2C Ports: 4
    - Digital Ports: 7(D2-D8)
    - Analog Ports(A0 - A3)
    - UART Ports: 1
    - Power Indicator: Green LED




Base Shield V2

- Justification:
    - Seeeduino LoRaWAN board compatibility: The Base Shield V2 is compatible with Seeeduino boards which made it ideal for this project.
    - It offered a cleaner prototyping workflow since Grove connectors could just be plugged in and used immediately.
    - The toggle switch support between 3.3 V and 5V made it versatile for various development platforms.

## Power: Battery & USB Power Bank

To power the Microcontroller board, Base Shied and sensors, a portable standalone **LiPo battery DT634169** and a back up USB power bank were used. Both products were rechargeable and compact which made them suitable for this IoT project.

- Features for LiPo Battery DTP6434169:
  - It is lightweight and easily portable.
  - It has Built-in Protection(PCB) against overcharge , overdischarge and sort-circuit among others.
- Specifications:
  - Nominal voltage: 3.7V
  - Charge Cut-off voltage: 4.2 V
  - Capacity: 2000mAh
  - Chemistry: Lithium-Polymer(Li-Po), cathode $LiNiMnCoO_2$


LiPo Battery DTP634169

- Justification:
  - This battery is compact and highly portable which made it efficient for the project.
  - It is safety prioritized due to the embedded PCB which provides essential protections and thus enhances safety and device integrity.

For backup power a Generic Power Bank with the following features was used:

- Cell capacity: 10000mAh 37wH
- Micro Input: 5V
- USB Output: 5V

## LCD Display: Grove 16x2 RGB LCD

This was used to display the readings during the data collection process. A green LCD was used due to its compatibility with the MC Board.


Grove LCD

## Communication Protocol: LoRaWAN & SWM

**LoRaWAN (Long Range Wide Area Network)** was used as the communication standard when transmitting the payloads over the network. It has the ability to transmit small payloads over long distances with very low power consumption, which is critical for IoT projects in urban areas like this one. It operates in unlicensed ISM frequency bands eg 868 MHz in Europe, 915 MHz in North America and allows for low-cot deployment without telecom licensing fees.

- LoRaWAN Device Classes
  - The project used **Class A** devices which is the most energy-efficient class. It follows n uplink-initiated communication model. Uplink where a node sends sensor data to the network server and a downlink where the gateway responds only after uplink windows.

**SWM & TTN** Integration was leveraged to provide a backend infrastructure for LoRaWAN. TTN connection was mostly strong around the central area of the city while SWM had better connection in the outskirts. Therefore, eventually SWM was used during the data collection because the study areas were in the city outskirts.

Core Functions of SWM in This Project:

- Gateway connectivity: SWM gateways received LoRa packets from the Seeduino LoRaWAN nodes which were then forwarded to the SWM Network Server, whereby they were decrypted and processed.
- Security and authentication: The IoT device (Thing) was registered on TTN and SWM with DevUI, AppEUI and AppKey which ensured that only authorized nodes joined the network. The end to end encryption protected payloads from tampering during transmission.
- Payload handling: SWM decoded Cayenne LPP payloads received form nodes. This data was the forwarded to external applications via a Webhook.

Advantages of LoRaWAN + SWM/TTN for this Project:

- City-wide coverage since SWM gateways can cover several kilometers.
- SWM and TTN  are free to use which makes them cost efficient and viable for a school project like this one.
- They enable low power operation which enables long term sensor deployment.
- LoRaWAN being an open standard enhances interoperability since it is compatible  with multiple sensor devices and cloud platforms

## Data Encoding: Cayenne Low Power Payload (LPP)

The Cayenne Low Power Payload is a standardized, binary-based data format that is designed specifically for low-power, bandwidth-constrained IoT networks like LoRaWAN. In this project, Cayenne LPP was used to encode sensor readings before transmission from the Seeduino LoRaWAN nodes to the SWM Network.

Motivation for Using Cayenne LPP

Cayenne LPP has strict limitations on the payload sizes - typically 51 bytes maximum. Sending raw JSON-formatted sensor data would easily exceed this limit. Therefore, Cayenne provides a lightweight alternatives such as :

- Flexibility in supporting multiple sensor data types.
- Wide support across IoT platforms
- Compactness through its binary encoding which reduces packet sizes.
- Interoperability by simplifying parsing on the server side.

## Middleware: FROST-Server

The system used the FORST Server which is an open source implementation of the OGC SensorThingsAPI standard.

**Entities Created and Managed**:

- Thing - Seeeduino LoRaWAN node
- Sensors: VEML 7700 , BME 680 and GPS Air 530
- Datastreams: Luminosity, temperature, pressure and humidity
- Observations: All individual sensor readings with time stamps.
- Locations: Geographic coordinates of the locations the Thing was at.

**Relevance of FROST Server in this  Project**

- It provided REST and MQTT endpoints for data access
- It offered standardized API and hence interoperability.
- FROST server was used to fetch data through real-time and historical queries.

### Data Persistence: MongoDB

All processed data and entities were persisted and stored in a MongoDB which was already set up and configured for use in this course. MongoDB being a NoSQL database optimized for time series and geospatial queries fit this project needs perfectly.

#### Database Design

The existing database design offered capabilities such as:

- Grouping collections by Things, Datastreams and Observations.
- Indexing the data by Time-series.
- Performing geospatial queries.

## Web Application

The web application *SafeWalkMunich* was majorly focused on user interaction. Its goal was to give a voice to the citizens by giving them the ability to drop pins on poorly lit areas, give feedback on general safety issues that they might have encountered while at the same time providing very useful data to the city and policy makers which can be acted upon and considered when prioritizing issues such as new street lamp installation. In addition, citizens can also see how illuminated the streets are and can maybe make safe navigation decisions if they want to, based on this.

### Overall concept and target users

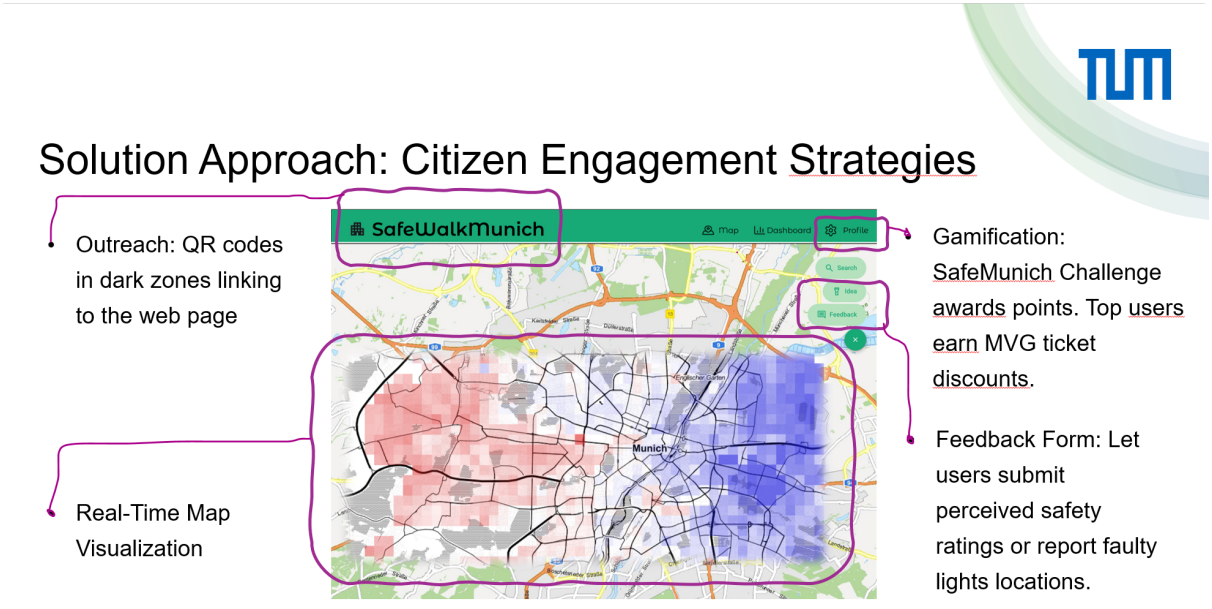
When the target audience is considered, quite a wide range of people is covered. Both pedestrians and cyclists are included, as well as residents who want to check how bright it will be along their route and how well-lit the road is within the route they plan to take. This is especially important and useful for vulnerable groups, such as women and children. It is considered important that citizens are given the opportunity and the voice to actually influence the development of urban infrastructure. The application can also be used by municipal services to identify where problems exist and what needs to be repaired.

The entire interface of the application is built upon several design goals. The first and most crucial is the clarity and openness of data, so that anyone can be given access to lighting information and can take it into account when planning their route in this context. For this purpose, in addition to the main functionality, an interactive dashboard providing an overview of all the data collected by the system has also been integrated. The second important milestone is engagement, achieved through various forms of voting and tools allowing comments to be left. A deep participation approach is applied, meaning that people are not simply considered a source of data, but those who can jointly generate ideas for the development of cities. And importantly, within this application, it is intended that numerical

sensor values be combined with people's subjective personal impressions — that is, mixed data — in order to obtain a more accurate picture.

## Overview of web components and reasoning

A key component of the interface is the Map View, which displays street lighting conditions with the ability to update the information in real time. The map itself includes several layers. One layer allows for more precise data reading — that is, it shows specific points where a particular lighting level was recorded at a certain moment in time, along with the corresponding environmental characteristics.
The second layer is more visual and appealing — it's the heatmap layer. It is still quite informative but may be slightly less accurate, as it represents an interpolation of values between the points in correlation between these interpolated values and color. Additionally, when zooming in or out on the map, the level of detail decreases or increases accordingly — for example, showing average values across a neighborhood when zoomed out. This approach makes the application more intuitive for users, as it allows them to quickly and easily read information in real time.
The map is equipped with several important features besides the map itself. The website includes a fairly advanced citizen feedback tool specifically for leaving comments, writing feedback, and participating in surveys.
Thanks to this, interaction with residents becomes possible, encouraging mutual influence between the people and the city and between the city and the quality of the urban environment in which residents live.




Moreover, since the project was originally intended to be developed in collaboration with SWM – Stadtwerke München, we considered (and still believe) that it is possible to add a user profile feature to the website in order to encourage participation in a gamified format. For example, users who leave more comments could win discounts on public transport tickets. All of this is clearly shown in the initial conceptual diagram, which was created using

Figma tools. Later, while working on the project, we decided to prioritize our effort to develop other features.

In addition, from the very beginning, we planned to publish our data in the form of a dashboard with open access, in order to ensure transparency and the possibility to analyze the data by city residents, researchers, and other users.

# 3D printing

## 3D printing technologies

As part of our project, it was necessary to create our own box that could hold all the components of our device inside and fit our specific tasks.
For this, it was necessary to study what 3D printing technologies exist in order to understand exactly how to implement our idea. Namely, there are several basic technologies depending on the material used and the goals of 3D printing. There are quite a few such technologies, but the three most popular ones are the following:

FDM/FFF is the most widespread and at the same time one of the oldest additive methods in the world. Its essence lies in the layer-by-layer application of melted material and the cooling of adjacent layers, which gradually merge with each other before the next layer is applied. To carry out such 3D printing, models are converted into special G-codes, which are sets of instructions. They are necessary in order to correctly position the drivers and, at the same time, ensure precise extrusion and creation of the next layer. A huge advantage of this technology is that it always uses a specific amount of material, but accuracy and detail may suffer somewhat.

SLS is a unique technology that consists of fusing polyamide particles using a high-energy laser beam. The process begins with filling the chamber with powder material; as the printing progresses, the working surface lowers, another layer of powder is added, and the sintering of this powder takes place carefully layer by layer. The advantage of this technology is that it does not require supports, since the model is naturally supported by the excess powder material. This technology allows the creation of geometrically complex elements with high precision; however, it is less accessible to the general public and quite expensive to implement.

SLA is an extremely precise 3D printing technology in which liquid photopolymer resin hardens under the influence of an ultraviolet laser beam. The material is stored in a reservoir into which the working platform is gradually immersed, and then the areas where the corresponding model should be are locally illuminated. The resin hardening process through illumination is repeated until the part is fully finished, after which it is washed in isopropyl alcohol to remove the uncured polymer. The disadvantage of this technology is that it is extremely expensive and also requires post-processing. After cleaning, the print is placed in a

special light-curing device, where resin models acquire their final properties. This technology is very good if smooth, flowing surfaces with a high level of detail are required.

After studying the possible technologies and also considering our financial capabilities, it was decided to use the most common approach — FDM. 3D printers of this type are available in the infrastructure of the Technical University of Munich. Therefore, all that remained was to design and prepare the model.

## Supported formats

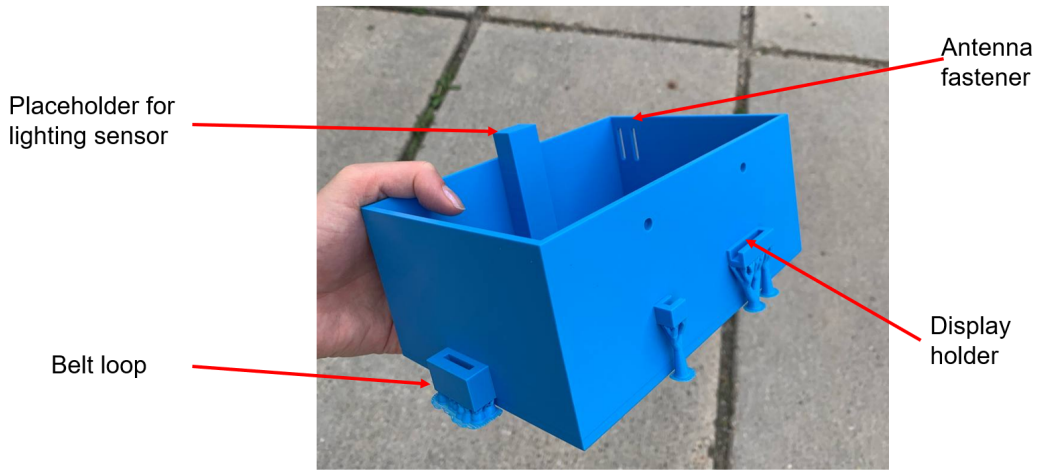
3D printers do not work directly with models from CAD software. Instead, models are exported to an intermediate format, which is then processed by a special slicer program and converted into the G-code format mentioned in the previous subsection.

STL is the most popular format, which describes geometry as a list of triangular faces and their normals. This format can be either text or binary. STL does not support colors or textures.

The second quite common format is OBJ. In general, it is similar to STL, but it supports textures and color. Since the final product's color was not of great importance to us and it was not planned to store this information inside our model, the STL format was chosen.

## Final solution

Although initially we considered modeling the box in Autodesk Revit, since for us it is the most familiar and accessible program, after preliminary research we decided to use Fusion 360 by Autodesk. This is a professional specialized program that combines tools for 3D modeling, designing various parts, analysis, and preparing files for production. It is widely used in the industrial design industry, mechanical engineering, and for creating any other 3D models for printing. The interface is quite convenient — for example, the grid is divided into one-centimeter cells, which is very practical because all the elements we needed to design within this project were also generally tied to centimeter measurements. For example, the display size. The logic of modeling is generally Boolean — meaning, for instance, a rectangular element is created, and then another volume is extracted from it, or one element is attached to another.




Hardware Holder

The hardware-holding box is equipped with special strap hinges, which allow it to be secured on the head using special Velcro straps that can wrap around the face like a helmet. Special openings are also provided so that the antenna can be fixed using the same type of straps. A display is installed on one side of the box, with holes provided for screwing it in place. On the opposite side, to prevent light leakage, a special mounting spot for the light sensor is installed.

However, some small drawbacks are also present in this design. During development, it was not fully taken into account that the actual size of the display might be several millimeters larger than specified on the website. Because of this, the display is not adequately secured on the special shelf designed for it, which, of course, makes the use of the device somewhat inconvenient.

The second problem is that the rather heavy box tends to tilt forward off the head during sudden movements. This issue could probably have been solved if a design more similar to a helmet had been made. However, that would likely have required a different budget for the project.

# Implementation

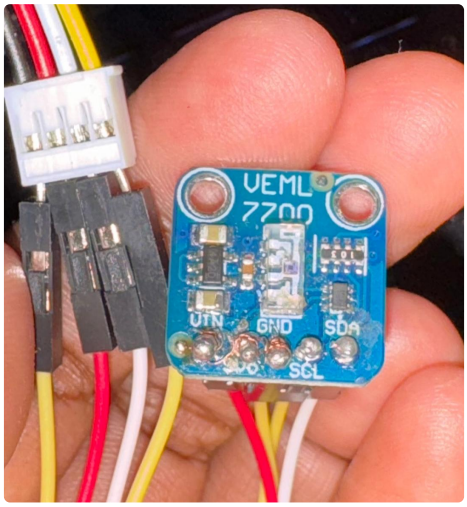
## Hardware Setup

**Sensors**

1. Grove BME 680
    - Directly compatible with Grove pins in Seeeduino LoRaWAN
    - Connected by plugging into I2C ports
2. Air530 GPS
    - Directly compatible with Grove pins in Seeeduino LoRaWAN

- Plugged into D4 port

3. VEML 7700
    - It is an adafruit product and hence not compatible with Grove interface
    - Header pins were soldered to breakout board. Header pins were connected to jumper wires that were connected to a Grove cable. See the figure below



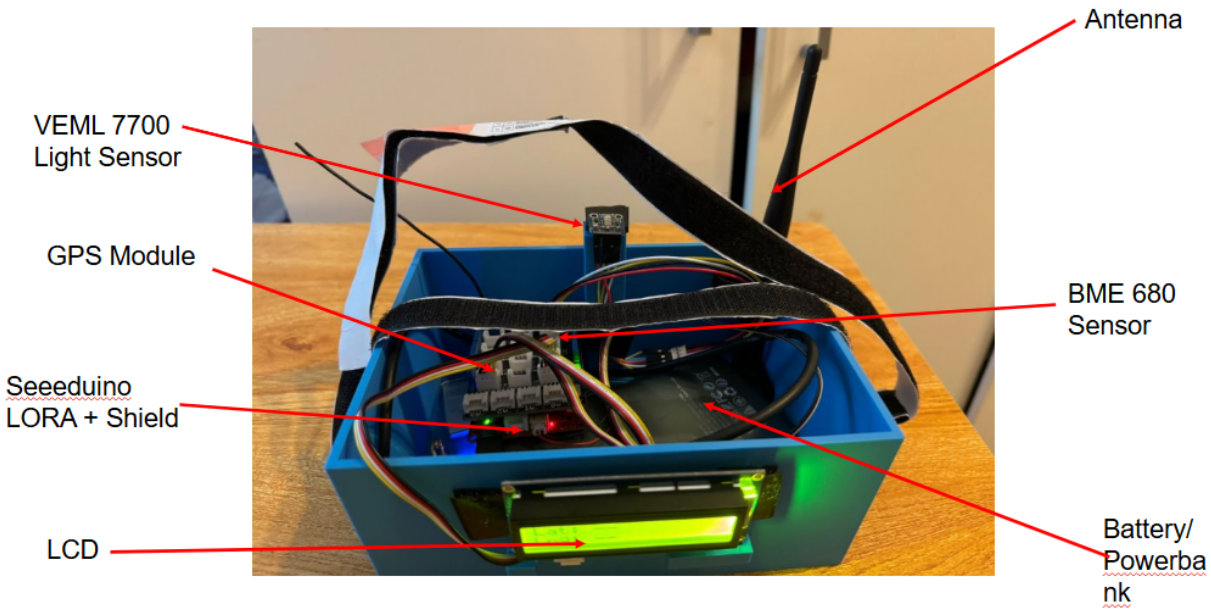

Soldered VEML 7700 Sensor With Jumper Wires

**Seeeduino LoraWAN MC Board**

- Connected via USB cable.
- Powered via LiPo battery and power bank.
- Expanded with Base shield v2.

**Assembly Steps**

1. Mounted Base Shield onto Seeeduino LoRaWAN MC female headers. This expanded the available I2C, UART and digital interfaces.
2. Prepared VEML 7700 by soldering male headers to breakout board, jumper wires connection which were letter connected to a Grove cable. This Grove cable was connected to Base Shield I2C
3. Connected Grove BME 680 to another I2C port on the Base Shield using grove cable.
4. Connected GPS Air 530 to D4 port on the Base shield
5. Connected Grove LCD screen to I2C port on the Base Shield.
6. Powered the system with LiPo battery and power bank for back up. During testing and programming USB cable was used.
7. Mounted the setup onto the 3D printed enclosure. Light sensor was mounted facing the streetlights in a clear unobstructed position. GPS antenna was exposed on top to ensure a sky view. The LCD Screen was placed on the outer face of the enclosure. See the figure below for the fully mounted setup.




Hardware Setup Inside 3D Printed Enclosure

## Sensor Calibration and Testing

**VEML 7700**

- **Reference**: Compared readings on the sensor and different lighting conditions to test sensor sensitivity and accuracy.
- **Method**:
  a. Collected readings outdoors in direct sunlight, indoors during the day, total simulated darkness, and outdoors under streetlights
  b. Compared the different readings and if they match expected results
- **Result**: Sensor read as high as 28000lux under direct sunlight, and as low as 0 lux in total simulated darkness. This proved that sensor readings reflected actual conditions and could be relied on.

**Grove BME 680**

- **Reference**: Ventusky Weather Live Radar for temperature, humidity and pressure.
- **Method**:
  - Recorded values at 10 minute intervals read from BME Sensor
  - Simultaneously manually recorded values from Ventusky web app at 10 minutes intervals at sensor location.
  - Compared the sensor readings to Ventusky web app values

- **Result**: Identified averagely 3.5% offset in humidity readings and 1% in temperature readings. This offset was not large enough to warrant adjustment in BME 680 readings, hence accuracy confirmed.

**GPS Air530**

- **Reference**: Smartphone Google Maps
- **Method**:
    - Placed both GPS Air530 and Smartphone at clear sky view at a  open outdoor location.
    - Logged coordinates simultaneously for 10 minutes.
    - Compared GPS module coordinates to Smartphone Google Maps coordinates values.
- **Result**: Observed very little to no deviation, less than 1m deviation. This confirmed that the Air530 GPS was accurate.

# IoT Integration

The IoT integration was done by modifying the provided source code to incorporate the different entities such as Seeeduino LoRaWAN library, Sensors and LCD  libraries, package the data and transmit it through the LoRa.

**Microcontroller Setup**

- Seeduino LoRaWAN was used as the central controller
- Grove Power Control was set up in Pin 33 as output to supply power to the sensors. The line was set HIGH at the startup to ensure all modules received power.
- Sensor Initilialization:
    - BME 680 was initialized using I2C address 0x76 in a loop with a 10 sec delay.
    - VEML 7700 was initialized over I2C
    - GPS Air530 was connected through SoftwareSerial pins 4(RX) and 5(TX). The GPS data was decoded using TinyGPS ++ library
- LCD Display:
    - It was initialized over I2C using the rgb_lcd library and it displayed initialization message in green

**LoRaWAN Setup**

- Identification: OTAA method was used with the device name, DeVUI, AppEUI and Appkey defined in code.
- Channels & Data Rate:
    - Five EU868 LoRaWAN channels were configure with adaptive data rate enabled for optimization.

- Power & Port: The transmission power was set to 14dBm, using port 33.
- Join Procedure: The OTAA join was set to attempt repeatedly until success with a delay of 1 second between retries

**Sensor Data Acquisition**

1. BME 680:
    - The temperature, humidity and pressure readings were read using the bme680.sensor_result_value.*property function*
    - The loop was set to return after every 30 seconds allowing for retry if reading failed
2. VEML 7700:
    - This sensor measured luminosity using the veml.readLux()
    - Similarly in case of failure then it would be retried after the 30 second loop
3. GPS Location:
    - The GPS data was read continuously for 2 seconds per loop iteration.
    - Latitude and longitude were added to the payload only after a valid fixed GPS was read.
    - GPS data was set using the functions gps.location.lat() and gps.location.long()

**Data Packaging**

- Cayenne LPP library was used here. Sensor readings were packaged into Cayenne Low Power Payload (LPP) format. using functions like lpp.addTemperature,
- Five channels ere set up, namely:
    - Channel 1 - Temperature(C)
    - Channel 2- Barometric pressure(hPa with scaling)
    - Channel 3 - Relative Humidity(%)
    - Channel 4 - GPS(lat/lng/alt)
    - Channel 5 - Luminosity(lux)

**Data Transmission**

- Payloads were sent using lora.transferPacket(). Each transmission took place once per loop iteration.
- For the downlink handling lora.receivePacke() function was called to check for any incoming packets.

The whole source code used for this setup is shown in the code block below.

**Implementation Source Code**

```
// Seeeduino LoRaWAN
//------------------------------------------------------------
#define PIN_GROVE_POWER 33

// LoRaWAN
//------------------------------------------------------------------------
---
#include <LoRaWan.h>

// Put your LoRa keys here
#define DevName "Gloria & Rita"
#define DevEUI "0064ECB6B5CE158C"
#define AppEUI "70B3D57ED002F952"
#define AppKey "A64E05519A14D9C945DF4223B4943AB6"

// CayenneLPP
//------------------------------------------------------------------------
#include <CayenneLPP.h>
CayenneLPP lpp(70);

// Sensor libraries
//------------------------------------------------------------
#include "seeed_bme680.h"
#include <TinyGPS++.h>
#include <SoftwareSerial.h>
#include <Adafruit_VEML7700.h>
#include "rgb_lcd.h"

#define BME_SCK 13
#define BME_MISO 12
#define BME_MOSI 11
#define BME_CS 10

#define IIC_ADDR uint8_t(0x76)
Seeed_BME680 bme680(IIC_ADDR);

TinyGPSPlus gps;
SoftwareSerial gpsSerial(4, 5);  // RX, TX pins

Adafruit_VEML7700 veml = Adafruit_VEML7700();

rgb_lcd lcd;

unsigned long lastSwitch = 0;
int screenPage = 0;

// SETUP
//------------------------------------------------------------------------
-----
void setup(void)
{
  pinMode(PIN_GROVE_POWER, OUTPUT);
  digitalWrite(PIN_GROVE_POWER, 1);

```

```
  gpsSerial.begin(9600);
  delay(100);

  while (!bme680.init()) {
    delay(10000);
  }

  if (!veml.begin()) {
    while (1) {
      Serial.println("Failed to find VEML7700 sensor!");
      delay(5000);
    }
  }

  lcd.begin(16, 2);
  lcd.setRGB(0, 255, 0);
  lcd.print("Initializing...");

  lora.init();
  lora.setId(NULL, DevEUI, AppEUI);
  lora.setKey(NULL, NULL, AppKey);
  lora.setDeciveMode(LWOTAA);
  lora.setDataRate(DR0, EU868);
  lora.setAdaptiveDataRate(true);

  lora.setChannel(0, 868.1);
  lora.setChannel(1, 868.3);
  lora.setChannel(2, 868.5);
  lora.setChannel(3, 867.1);
  lora.setChannel(4, 867.3);
  lora.setChannel(5, 867.3);

  lora.setDutyCycle(false);
  lora.setJoinDutyCycle(false);
  lora.setPower(14);
  lora.setPort(33);

  while (!lora.setOTAAJoin(JOIN, 20)) {
    delay(1000);
  }
}

// LOOP
----------------------------------------------------------------------
void loop(void)
{
  if (bme680.read_sensor_data()) {
    delay(30000);
    return;
  }

  float lux = veml.readLux();

  lpp.reset();
  lpp.addTemperature(1, bme680.sensor_result_value.temperature);
  lpp.addBarometricPressure(2, bme680.sensor_result_value.pressure /
1000.0);
  lpp.addRelativeHumidity(3, bme680.sensor_result_value.humidity);
```

```
    lpp.addLuminosity(5, (uint16_t)lux);

    unsigned long start = millis();
    while (millis() - start < 2000) {
      while (gpsSerial.available()) {
        gps.encode(gpsSerial.read());
      }
    }

    if (gps.location.isValid()) {
      float lat = gps.location.lat();
      float lng = gps.location.lng();
      lpp.addGPS(4, lat, lng, 0.0);
    }

    lora.transferPacket(lpp.getBuffer(), lpp.getSize(), 5);

    short rssi;
    char rx[256];
    short length = lora.receivePacket(rx, 256, &rssi);

    if (millis() - lastSwitch > 4000) {
      lastSwitch = millis();
      screenPage = (screenPage + 1) % 2;
      lcd.clear();

      if (screenPage == 0) {
        lcd.setCursor(0, 0);
        lcd.print("Light:");
        lcd.print(lux, 0);
        lcd.print("lx");

        lcd.setCursor(0, 1);
        lcd.print("T:");
        lcd.print(bme680.sensor_result_value.temperature, 1);
        lcd.print("C H:");
        lcd.print(bme680.sensor_result_value.humidity, 0);
        lcd.print("%");
      } else {
        lcd.setCursor(0, 0);
        if (gps.location.isValid()) {
          lcd.print("Lat:");
          lcd.print(gps.location.lat(), 6);
        } else {
          lcd.print("Lat: --");
        }

        lcd.setCursor(0, 1);
        if (gps.location.isValid()) {
          lcd.print("Lng:");
          lcd.print(gps.location.lng(), 6);
        } else {
          lcd.print("Lng: --");
        }
      }
    }

    delay(30000);
```

```
162 }
```

# Cloud Setup

## FROST Server : Entity Creation

FROST Server was used to create and manage the entities used in this project. They were all created using a JSON body input and POST method over the FROST Server. Entities created include:

- Thing - which represents the Seeeduino LoRaWAN microcontroller.
  - The entity description contained metadata such as name, description, encoding type and location coordinates.
  - See code block below to see the thing created and used in this project.

**Thing Entity Creation**

```
{
  "name": "Group 4 Thing",
  "description": "Testing lighting across Munich, gps and environmental data.",
  "properties": {
    "frequency": "30 s",
    "groupNo": 4,
    "DevEUI": "0064ECB6B5CE158C"
  },
  "Locations": [
    {
      "name": "Trudering, street add - Testlocation",
      "description": "Test location",
      "encodingType": "application/vnd.geo+json",
      "location": {
        "type": "Point",
        "coordinates": [
          11.694171,
          48.113087
        ]
      }
    }
  ]
}
```

- Sensors - Three entities were created to represent the sensors . The name, description, encoding type and metadata were defined for each.
  - VEML 7700 - Luminosity
    - **VEML Sensor Entity**

```
{
  "name": "Adafruit VEML 7700 Ambient Light Sensor",
  "description": "VEML 7700 Light Sensor",
  "encodingType": "text/html",
  "metadata": "https://learn.adafruit.com/adafruit-veml7700/overview"
}
```

  - BME 680
    - **BME Sensor Entity Creation**

```
{
  "name": "Grove BME680",
  "description": "Grove Temperature, Humidity, Pressure & Gas Sensor v1.0",
  "encodingType": "text/html",
  "metadata": "https://wiki.seeedstudio.com/Grove-Temperature_Humidity_Pressure_Gas_Sensor_BME680/"
}
```

  - GPS Module
    - IoT Id:
    - **GPS Air530 Entity Creation**

```
{
  "name": "Grove GPS(Air 530)",
  "description": "Grove GPS  v1.0",
  "encodingType": "text/html",
  "metadata": "https://wiki.seeedstudio.com/Grove-GPS-Air530/"
}
```

- ObservedProperties - these define all the measurable properties : Illuminance, Temperature, Relative Humidity, Pressure, Latitude & Longitude. Five observed properties entities were created:
  - Temperature

- IoT Id: 480
- **Temperature ObservedProperty Entity**

```
{
    "name": "Temperature",
    "definition": "https://wiki.seeedstudio.com/Grove-Temperature_Humidity_Pressure_Gas_Sensor_BME680/#Temperature",
    "description": "Temperature Readings."
}
```

- Relative Humidity
  - IoT Id: 481
  - **Relative Humidity ObservedProperty Entity**

```
{
    "name": "Humidity",
    "name": "Humidity",
    "definition": "https://wiki.seeedstudio.com/Grove-Temperature_Humidity_Pressure_Gas_Sensor_BME680/#Humidity",
    "description": "Humidity Readings."
}
```

- Pressure
  - IoT Id: 482
  - **Pressure ObservedProperty Entity**

```
{
    "name": "Pressure",
    "definition": "https://wiki.seeedstudio.com/Grove-Temperature_Humidity_Pressure_Gas_Sensor_BME680/#Pressure",
    "description": "Pressure Readings."
}
```

- GPS
  - IoT Id: 479

- **GPS ObservedProperty**

```
{
    "name": "GPS",
    "definition": "https://wiki.seeedstudio.com/Grove-GPS-Air530/#GPSCoordinates",
    "description": "GPS Readings."
}
```

- Luminosity
  - IoT ID: 478

- **Light ObservedProperty Entity**

```
   {
    "name": "Light",
    "definition": "https://wiki.seeedstudio.com/Grove-Digital_Light_Sensor/#ThermodynamicTemperature",
    "description": "The Light."
   }
```

- Datastreams - Datastream entities represent a collection of observations of a specific ObservedProperty for a particular Thing using a specific sensor. Five datastreams were made to represent the five observed property. GPS datastream was created in this case instead of solely relying on the locations because of the downstream use in the web application where GPS and lux readings needed to be combined to create a GeoJSON file. Using the GPS datastream proved easier to fetch from and process.
  - Two datastreams examples will be provided here to show a glimpse of how the datastreams were created. The rest were created in a similar manner. The properties included include: name, description, unit od measurement, Thing id, sensor id, observed property id among others.
  - GPS: Linked to the GPS AIr530 module, The Thing and the GPS ObservedProperty

- **GPS Datastream**

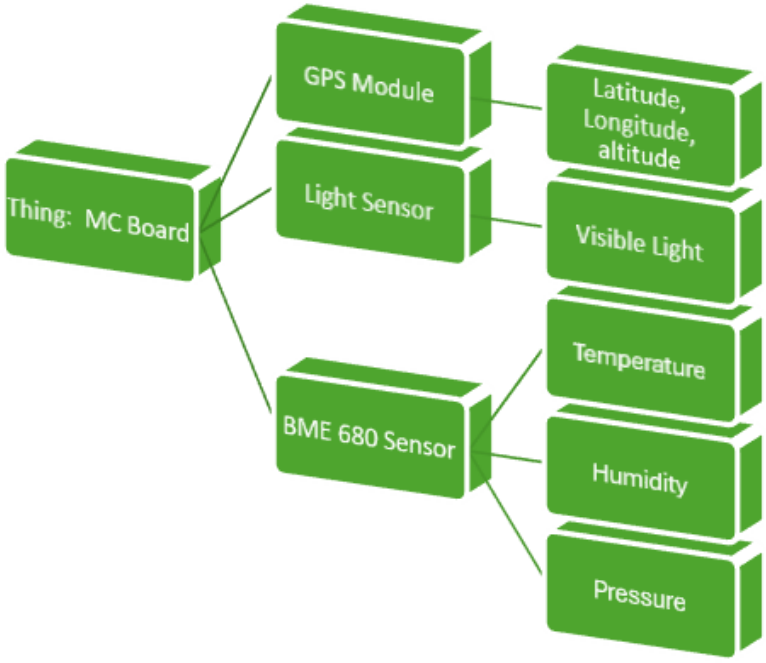
```
{
  "name": "GPS associated with Munich City Night-Lighting: A small area + environmental data",
  "description": " The GPS readings of every measured light values",
  "observationType": "http://www.opengis.net/def/observationType/OGC-OM/2.0/OM_Measurement",
  "unitOfMeasurement": {
    "name": "Decimal Degrees",
    "symbol": "DD",
```

```
    "definition": "http://www.qudt.org/qudt/owl/1.0.0/unit/Instances.html#DD"
  },
  "Thing": {
    "@iot.id": 572
  },
  "ObservedProperty": {
    "@iot.id": 479
  },
  "Sensor": {
    "@iot.id": 594
  }
}
```

- Luminosity: It was tied to the VEML 7700 sensor, The Thing(Seeeduino LoRaWAN) and the Luminosity ObservedProperty

- **Luminosity Datastream**

```
{
  "name": "Munich City Night-Lighting: A small area",
  "description": " The light intensities in streets at night",
  "observationType": "http://www.opengis.net/def/observationType/OGC-OM/2.0/OM_Measurement",
  "unitOfMeasurement": {
    "name": "Illuminance",
    "symbol": "Lux",
    "definition": "http://www.qudt.org/qudt/owl/1.0.0/unit/Instances.html#Lux"
  },
  "Thing": {
    "@iot.id": 572
  },
  "ObservedProperty": {
    "@iot.id": 478
  },
  "Sensor": {
    "@iot.id": 595
  }
}
```



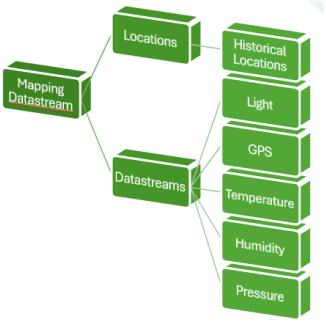




Illustration of Entities and Datastreams Created

## MongoDB Mapping

MongoDB was used to store sensors data for the project. The schema was organised into two primary collections namely Things and Observations. These collections captured the metadata data about the Things and the Observations.

Things Collection

- Stored information about Seeeduino LoRaWAN MC used. For instance the id and the locations information.

Observations Collection:

- Stored individual sensor measurements observed. Each datastream with its associated obesrvation were mapped. For each single observation, its associated time stamp were added into their datastream during recording. Based on the datastream the defined unit of measurement was also attached to each observation.

**MongoDB Mapping Document**

```
{
    _id: '0064ECB6B5CE158C',
    name: 'Group4: IoT for Citizen Engagement',
    group_id: 4,
    dev_eui: '0064ECB6B5CE158C',
    datastreams: [
        {
            lpp_id: 1,
            sta_servers: [
                {
                    sta_url: 'https://gi3.gis.lrg.tum.de/frost/v1.1',
                    datastream_iot_id: 1504
                }
            ]
        },
        {
            lpp_id: 2,
            sta_servers: [
                {
                    sta_url: 'https://gi3.gis.lrg.tum.de/frost/v1.1',
                    datastream_iot_id: 1506
                }
            ]
        },
        {
            lpp_id: 3,
            sta_servers: [
                {
                    sta_url: 'https://gi3.gis.lrg.tum.de/frost/v1.1',
                    datastream_iot_id: 1505
                }
            ]
        },
        {
            lpp_id: 4,
            sta_servers: [
                {
                    sta_url: 'https://gi3.gis.lrg.tum.de/frost/v1.1',
                    datastream_iot_id: 1503
                }
            ]
        },
```

```
        {
            lpp_id: 5,
            sta_servers: [
                {
                    sta_url: 'https://gi3.gis.lrg.tum.de/frost/v1.1',
                    datastream_iot_id: 1502
                }
            ]
        }
    ],
    locations: [
        {
            lpp_id: 4,
            sta_servers: [
                {
                    sta_url: 'https://gi3.gis.lrg.tum.de/frost/v1.1',
                    thing_iot_id: 572
                }
            ]
        }
    ]
}
```

## Sensor Data Planning Table

Below is a summary of all the entities and mappings done for this project.

| Names | Group No | Thing | Sensor | ObservedProperty | LPP Channel Nr | FROST Datasream ID |
|---|---|---|---|---|---|---|
| Gloria Asenath Kiawa<br><br>Margarita Zykova | 4 | Seeeduino<br><br>LORAWAN mobile<br><br>DEVUI: 0064ECB6B5CE158C/<br>8765182202E81E0A | Grove Air350GPS | Location[DD] | 4 | 1503 |
| | | | BME 680 | Temperature [C] | 1 | 1504 |
| | | | | Humidity [%H] | 3 | 1505 |
| | | | | Pressure [KPa] | 2 | 1506 |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | Adafruit VEML 7700 | Visible Light [lux] | 5 | 1502 |

## Web Application Development

A full-scale citizen participation platform was aimed to be created, built on a client–server architecture. This application is composed of two key layers — Backend (server) and Frontend (client). A separate Backend server has been set up to run on its own port 5000, and it has been integrated with a MongoDB database. The Backend is used for processing requests, interacting with the database, and returning responses, while the Frontend is used for receiving and displaying the data.

*SafeWalkMunich* is provided with a range of core functions. These include an interactive map with various layers, such as heatmap-style lighting visualizations, the display of sensor data from different points, a dashboard, and a variety of engagement tools. More details on each of these will be provided in the respective subsections.

### Frontend

#### Stack

For our front-end architecture, the JavaScript framework React was used, which provides a number of advantages. These include modularity, functional components with hooks, and the Context API for global state management. The React Context API is a built-in state management system that allows data to be shared between different components of the system without the constant passing of properties through all levels. Functions and data that need to be made available to many components are defined in a context provider component and passed through its value property; they can then be accessed in any child component by using the useContext hook.

For building, the development server Vite was used, allowing the development server to be launched instantly and supporting configuration on port 3000. Routing was done using React Router DOM. This is a library for client-side routing that determines which React components to display when the URL changes, thereby controlling the display of pages without completely reloading the site. If data is important for many parts of the application at once, for example, user authorisation, it is convenient to store it in a context file. This greatly simplifies synchronisation because when data changes in the context, all components that use it automatically receive updated values.

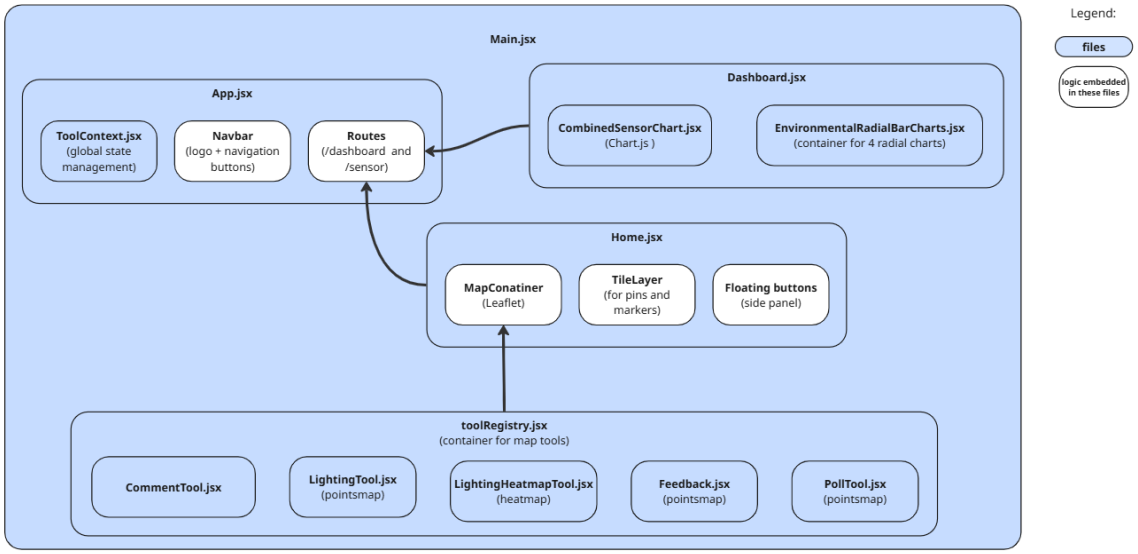
For data visualization, several libraries were used, namely Chart.js — the main library that allows the creation of line charts for time series, radial charts, environmental indicators, and more. It has an adapter for working with dates — chartjs-adapter-date-fns, which enables Chart.js to work correctly with dates using functions from date-fns. date-fns is a standalone library for date manipulation. ApexCharts was also used as an additional library for more advanced charts and alternative types of visualization with interactive controls.

When it comes to cartography and geospatial data, we used the Leaflet library. It supports a variety of map providers, GeoJSON data, and various interactive controls. To create a heat map, we used Leaflet.heat, which allows any type of data and indicators to be visualised in the format of such a heat map. And we made a React wrapper for Leaflet to integrate maps with React components and manage their state within the application architecture.

To run development tools on the frontend, Node.js was used together with Vite. We also used npm — Node Package Manager. It is a package manager that installs all dependencies listed in the package.json. With its help, React, Vite, Chart.js, and any other libraries were installed on the frontend. npm also manages the node_modules folder, where all dependencies are stored. However, the actual frontend code — that is, React, JavaScript, and CSS — is executed in the browser, not in Node.js. So, Node.js was used only for development tools on the frontend.

## Key Features

As mentioned earlier, the JavaScript frontend framework React was used, which allows a modular structure to be created for web interfaces. Thanks to this, the project was divided into logical components, and reusable tools were declared using Tool Context, enabling centralized state management for frontend elements. Rendering was optimized, and large datasets were handled efficiently through React, which also ensured support for different screen sizes and responsive design.




Overview of main functional components in the frontend architecture

When it comes to the component structure, the main root component of the application is App.jsx, which is responsible for the context (via ToolProvider), routing, and navigation — namely, the top panel with the logo and navigation buttons.

ToolContext.jsx is the component made possible specifically through the use of React's Context API for managing global state. As for state, for example, active tools and the management of the array of pins on the map are declared in this context file, along with several other functions — such as addPin and setActiveTool.

If the two main pages are considered, we have Home, the main page with the map, and Dashboard, the page with the charts that reflect the state of data and information. Home includes MapContainer, a map based on Leaflet, and also TileLayer, which provides OpenStreetMap tiles. It also includes dynamically rendered tools from the toolRegistry. All key functionalities — such as the ability to leave comments, the lighting points map, the heatmap, feedback, and polling tools — are connected to MapContainer in Home.jsx via toolRegistry. Home.jsx also includes FloatingButtons, essentially a side panel with functions that allow users to activate any available map tool.

As for Dashboard.jsx, it consists of two key components. The first is CombinedSensorChart.jsx, where environmental and lighting indicators are displayed simultaneously, with filtering by time and date. The second is EnvironmentalRadialBarCharts.jsx, a container for four radial charts, each representing different data over different time periods.

### Design Approach

An adaptive interface with a responsive layout was aimed to be created, which could work with different screen sizes and adjust accordingly. It is not yet perfectly displayed in the mobile app format, but the potential for that is present.
As for the design, the Lucide React icon library was used. It includes a large collection of modern SVG icons, supporting various colors and sizes. The font system Montserrat was chosen — a fairly popular classic font currently available on Google Fonts.
Several CSS modules were created to define styles. App.css and index.css were used for the global styles of the application. Separate files Dashboard.css and EnvironmentalRadialBarCharts.css were also added in order to avoid several developers having merging conflicts while adjusting the same style files for the frontend. Since the CombinedSensorChart.jsx has a fairly simple structure, a separate CSS file was not created for it, and the style was declared directly in the file.
But the EnvironmentalRadialBarChart.jsx is a somewhat more complex component, which consists of four other components — one for each radial chart. Because of this, a separate CSS file was allocated.

## Backend

The backend layer in our application is focused on handling requests from the frontend via the REST API, interacting with the database through specific models, and processing user-submitted comments, feedback, and polls — in other words, implementing the business logic for these tasks.

### Stack

As already mentioned in the introduction to the web development section, the project has been structured into both frontend and backend parts. The backend is responsible for the server-side logic of the project, including business logic implementation, database operations, and handling requests from the frontend. In this section, the technology stack will be reviewed.

In order to facilitate communication between the frontend and backend, we decided to use Node.js for backend server development in addition to our JavaScript-based frontend stack. Node.js is the most popular JavaScript execution environment and offers asynchronous request processing and high performance. It gives access to the file system, network requests, streams, and other essential operating system features, enabling JavaScript to be used in the browser and for server-side development.

Express, a framework for the Node.js environment, was applied to implement all these backend features. In this project, it is used for routing, handling HTTP requests and responses, and connecting middleware, such as CORS and JSON parsing. To set up data storage, MongoDB and Mongoose were implemented. MongoDB is a document-oriented

NoSQL database that stores project data such as comments, feedback, survey responses, etc. Mongoose is an ODM (Object Data Modeling) library for Node.js, used to manage MongoDB operations. It enables data schemas to be defined and validated.

As already noted, middleware is connected to support web application functionality. For example, CORS middleware (Cross-Origin Resource Sharing) was used — a mechanism that allows or restricts access to server resources from other domains. This is necessary when the frontend and backend are hosted on different domains or ports, as in our case.

To securely handle the Mongo URI for connecting to the MongoDB database, the dotenv library was used, which helps manage .env files containing this information. It works as follows: the Mongo URI, which includes the address of the MongoDB server, the name of the database, and sometimes a login and password, is used to establish a connection with the Mongoose library, which was mentioned earlier.

This allows for secure storage and automatic loading of private information when connecting to the database, avoiding the need to declare this link directly in the source code. It helps protect the information on the site's server, which will later be made publicly accessible. Since we plan to store user comments in MongoDB and want this to be done securely — making sure not every user has access to this data — this method of encrypting the database link is used.

In addition, this is a convenient approach if switching to another server is required later. In that case, it would only be necessary to replace the Mongo URI in the .env file, with no changes needed to the source code, which is very convenient.

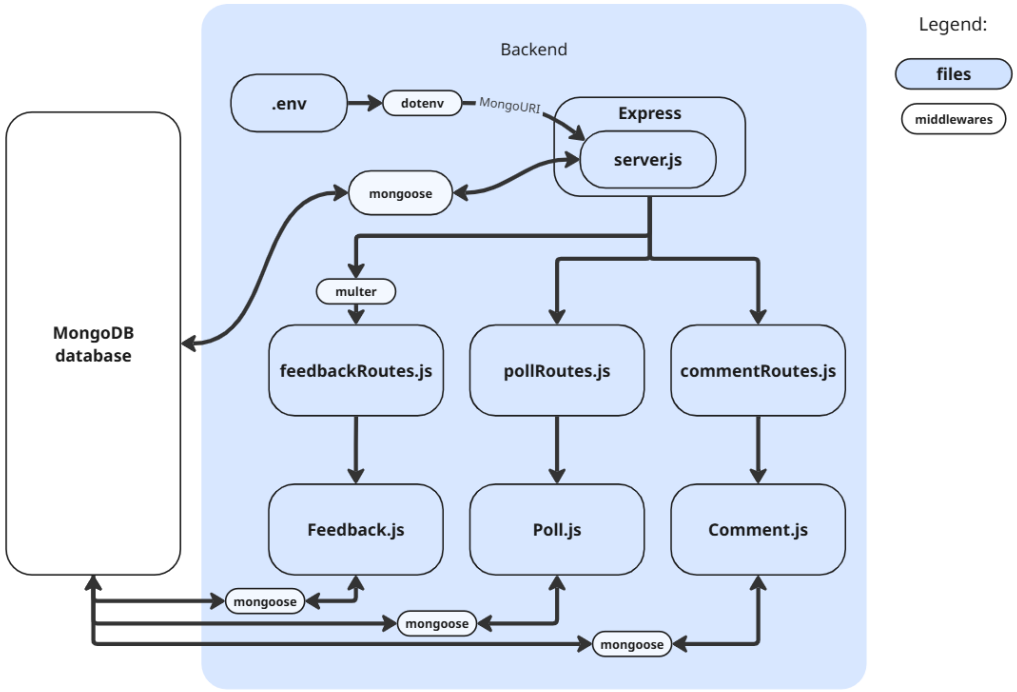
Another additional tool used in the project is the multer middleware library. It is designed to process multipart/form-data requests, which are typically used to upload files to the server, such as images, documents, and so on. In particular, in this project, multer is used to process text data submitted through feedback forms. It ensures a basic setup for forms.

Although in the frontend Node.js serves only to launch the development server, in the backend, Node.js is a full runtime environment for executing server-side code using Express.js, server routes, and working with the database. In the backend, npm also manages dependencies — but of course, these are different dependencies: Express, Mongoose, dotenv, and so on. And the actual backend server code is executed in Node.js.

## Backend architecture

The main configuration and server startup file, through which interaction with the backend is initiated, is server.js. In this file, the Express server is configured — routing is enabled, route files are connected, and specific middleware is set up. Once the interaction process with the backend server is started, access to route files is provided through various endpoints. These route files define how HTTP requests to different parts of the API are processed. In the backend part of the project, three main routes have been implemented: for handling

comments, for collecting and processing feedback, and for managing polls.
Middleware like body parsing and Multer for file uploads are also set up in server.js.




Interaction between backend modules, middleware, and database

After the interaction with the route files is established and the HTTP request is processed, the connection to the data model files is made. These files define the structure of collections (database tables) and the methods for interacting with them.

In this project, three models have been implemented — one for each type of user input: a MongoDB model for handling comments, a Feedback model for storing and processing user feedback, and a Poll model for handling surveys and voting. Interaction with the MongoDB database is carried out through these models using the Mongoose ODM Library.

## Collaboration and Management tools

GitLab was used for collaborative work. It was used as the main platform where the code was stored and the change history was maintained throughout the development process of the web application.
The branching scheme was kept relatively simple: a main branch, Main, was used primarily by Gloria, and a separate branch, Zykova, was created for the second developer, where development was carried out.
Later, a Merge Request was used to merge changes from this branch with changes from the Main branch. This allowed the participants to review code, leave comments, and reduce the risk of errors.

In this context, it's worth briefly explaining how GitLab — or any other Git version control system — works. First of all, before starting work, the project is copied locally from the cloud storage to the computer. During the process of working on the project, it is necessary to regularly fix the amount of work done locally — in other words, every step of the work that has been completed. This is called a commit. It divides the work into understandable sub-steps and allows rolling back to a previous version if needed. When making a commit, it's important to write a short and clear message explaining what exactly is included in that saved step.

The Push function is the action of sending commits to GitLab — to the server, into the remote repository. This allows other developers to access the updated version and continue working on the project. There is also the important Pull function — updating the local branch with changes from the server. This should be done regularly to reduce the chances of conflicts during collaborative work.

In addition, for collaboration, we used WhatsApp for quick communication, Miro Board for project management (Kanban), as well as for diagramming and collecting information. On top of that, we also used Figma to create the first mockup of our project, which was mentioned earlier in "Web Application" section.

Link to GitHub https://github.com/gloriak9/IoT-for-Citizen-Engagement-in-Smart-Cities

## Web Application Deployment

To enable wide access of the web application, it was deployed in distributed hosting services to ensure that it could be accessed across the web. Since the web app contained both a frontend and backend they were deployed separately and linked to ensure a persistent connection between the web interface and the MongoDB used for data storage.

**Frontend Deployment:** Here Netlify which is web development and hosting was proffered because it is open and free to use and it can be directly connected to the Github repository for quick deployment and development if changes are needed.

**Backend Deployment**: Render was used here because it is free to use, automatically redeploys the app upon changes in the base code. Additionally it also offers Github integration. MongoDB was also integrated as part of the setup during this deployment to ensure persistent data storage.

**Frontend and Backend Integration**: the frontend was integrated into the backend using CORS, whereby the new web app endpoint URL was set as the origin. The backend URL from deployment was added to the front end pages that need to post api calls into the backend.

**Testing and validation**: the deployed app was tested by sharing the URL to some users around Munich who were asked to provide appropriate feedback on some safety issues experienced. Their feedback was found to be stored in the database and could be pulled by GET api methods. From this it was concluded that the web app was fully functional and met

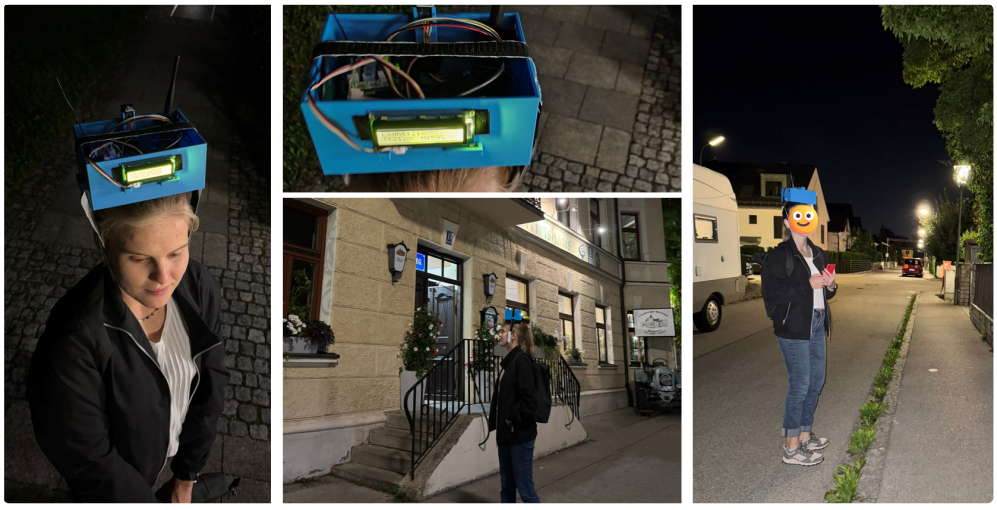
the need of giving the user a voice and providing the policy makers with a mine of useful data.
Link to our Web site https://munichsafewalk.netlify.app/

## Testing the Complete System - Data Collection

To test out the complete system, we went out in the city for a long 3 hours walk. During the testing of the system under real-world conditions, no significant issues were identified. The antenna signal was found to be sufficiently strong, no overheating of components was observed, and the battery life — the system was charged via a power bank — was sustained for the full 3 hours of the walk, which is considered a fairly good result.

**Data Collection**:

1. The system was setup and checked to ensure that everything was operating properly.
2. The NODE-Red endpoint was checked to ensure payloads were being transmitted before the actual data collection.
3. The complete system was then mounted on the head as shown in the figure below for proper light sensor exposure and to limit possible data noise form headlights from oncoming cars.
4. 3 cycles around the study area were made, the walk was made on the street side with street lights.



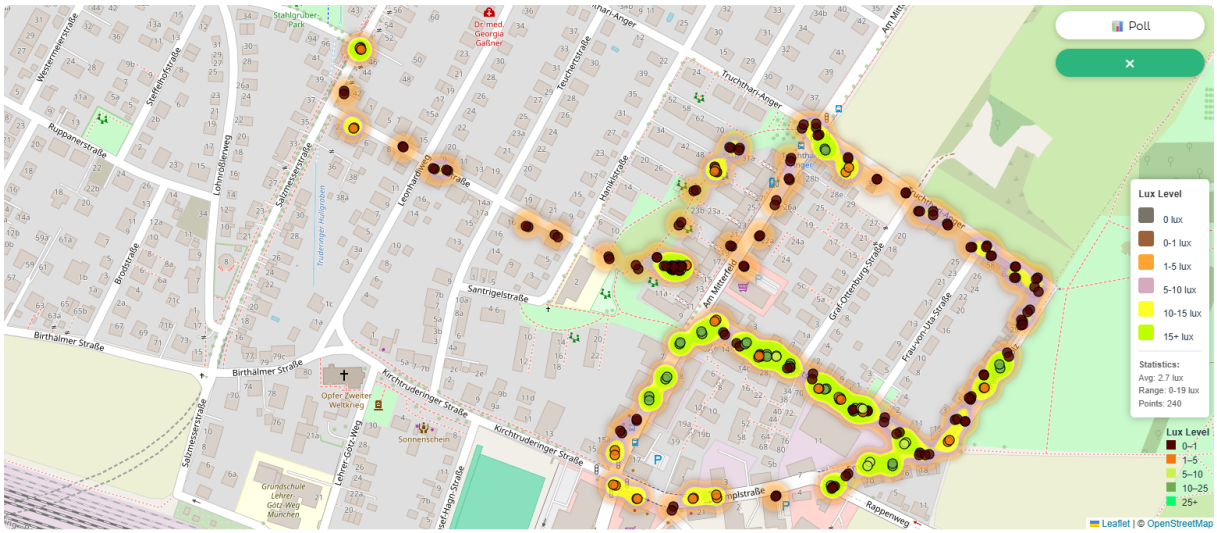
Photographs of data collection and fields measurements

# Results and Evaluation

## One-dimensional analysis of lighting

The key parameter of the study is the illumination level, so a one-dimensional analysis of its distribution along the route is presented below. As part of the research, an urban area in the Moosfeld district was examined, including residential blocks, green zones, and street infrastructure. This was done during a walking route from 9 p.m. to approximately 11:40 p.m.

Brightly lit areas are mainly concentrated around intersections and public transport stops. Poorly lit areas include inner courtyards, pedestrian paths, and partly green zones, which may be potentially unsafe during evening hours. Despite the low lux levels, some areas within the residential environment are not perceived as unsafe. This can be explained by high population density, sound permeability, and active evening use.




Composite map view integrating both heatmap and point-based illuminance data from our self-developed web tool

A second important aspect is that the data we collected shows an uneven distribution of street lighting, which may indicate a need for redistribution of lighting infrastructure or vegetation maintenance, since poor lighting can also be caused by natural obstructions, such as trees partially blocking the street lamps. In general, all of this validates our project as a source of information about where lighting in the city needs to be improved, and it may serve as a useful tool for urban authorities, signaling the need for changes in the urban environment.

## Complex analysis of measured data

A particular difficulty for a comprehensive analysis of the measured parameters was the fact that we made several loops around the district, the area, and because of that, some of the data overlapped, with different time parameters. On the one hand, this was done intentionally to avoid incorrect calculations and to be able to determine the average value. However,

unfortunately, it also made the process more complicated, because it was very difficult to correlate time frames, lighting levels, and the locations where certain data characteristics were collected.

In this table, the key metrics that we found to be the most significant are presented. The data is sorted according to the key areas of our route. Since, overall, the values of barometric pressure did not change much during the entire data collection period, this metric is not included in the table. The correlation between temperature and humidity is discussed in a separate subsection.

**Key metrics summary table**

| Time period | Illumination | Temperture | Safety feeling | Route area | Comment |
|---|---|---|---|---|---|
| ~ 21:25 & ~22:20 | 11 → 4 lux | 24 → 23 °C | Safe | Am Mitterfeld (park) | Natural lighting is still partially represented (twilight) |
| ~ 21:40 & ~ 22:30 | <2 lux | ~21 °C | Unsafe | Truchthari-Anger Straße | Dark area, beginning of fluctuations |
| ~ 21:35 & ~22:40 | <2 lux | ~18.3 °C | Unsafe | Straßl ins Holz (field) | The darkest and coolest area |
| ~ 22:05 & ~ 22:40 & ~23:20 | 10-15+ lux | ~20 °C | Safe | Karotschstraße | Calm residential street, well-lit |
| ~22:50 | <2 lux | ~18.3 °C | Safe | Am Mitterfeld between Karotschstraße and Truchthari-Anger Straße | There are lamp posts there, but too scattered |
| ~23:00 | ~8 lux | ~18 °C | Safe | Am Mitterfeld towards Schmuckerweg | Commercial services in this area |
| ~23:10 | 19 lux | ~19.5 °C | Safe | Emplstraße | Poorly vegetated urban area |
| ~ 23:50 | 0–3 lux | 18 °C | Moderately safe | Ilmstraße | Moderately dark, chilly |

If we look at our indicators comprehensively, several key observations can be made. Let's follow them step by step. We started our route near the residential area at Am Mitterfeld 15–17 and began moving towards Truchthari-Anger. We reached that area around 9:30 p.m. At

that time, a gradual decrease in illumination was observed, as there was still natural twilight, but the street lights were already functioning and contributing to the overall lighting level. Nevertheless, the graph shows a relatively smooth decrease until around 10 p.m. During this time, the temperature also began to slowly decline by a few tenths of a degree.

Next, we continued from Truchthari-Anger toward Straßl ins Holz. In general, this street is rather poorly lit and is located near agricultural fields. These areas also show low temperature levels and slightly higher humidity values. As previously mentioned, these segments were subjectively identified by us as the least safe.

Overall, the areas of Karotschstraße and Am Mitterfeld towards Schmuckerweg are relatively calm and safe, and are fairly well lit. The same can generally be said about Emplstraße, which is also the least green segment of our route.

On the way back to the subway, we also made some measurements of the route along Ilmstraße. Different segments of this street are lit to varying degrees — some of them are somewhat darker — but overall, the area feels calm and quiet. The overall sense of safety was maintained due to residential development and the possibility of being observed by local residents.

However, at the intersection of Salzmesserstraße and Ilmstraße, an unpleasant social episode occurred — a group of unfamiliar teenagers approached us in a car and started asking questions. This situation did not feel entirely comfortable or safe, despite the level of illumination. For this reason, the segment is marked as moderately safe.

The segment of Am Mitterfeld between Karotschstraße and Truchthari-Anger Straße raised some confusion due to its measurements, since there are clearly installed street lamps there. However, according to our readings, the illumination levels were mostly very low. A similar situation happened with Ilmstraße.

Facade view of the house Am Mitterfeld 16. Shows that at Karotschstraße the lamp posts are lower.

One possible explanation might be the high walking speed, which could have affected the data sampling density and the height of the lamp posts. This aspect is pretty interesting in the context that the height of lamp posts influenced the author's subjective feeling of safety. The street "Am Mitterfeld" is really dark at night. Even though our data about this particular road segment is not entirely accurate — and lighting is in fact present and installed — when it comes to the subjective perception of comfort and lighting levels, the data is still indicative and relevant for use in analyzing urban infrastructure.

Night view of the segment Am Mitterfeld between Karotschstraße and Truchthari-Anger Straße

Night view in the one of inner courtyards near the park (Am Mitterfeld)

Additionally, in the residential zone of Am Mitterfeld (inner courtyards near the park), the lighting is provided by low-height lamp stands (approximately 900 mm tall), which makes it difficult for the sensor — mounted at head level — to adequately register these light sources.

## Correlation between temperature and humidity: complex analysis

During the comprehensive analysis of the collected data, a clear inverse correlation was identified between changes in air temperature and relative humidity levels. In the presented graphs, it is visible that local peaks in humidity correspond to drops in temperature, and vice versa — an increase in temperature coincides with a decrease in humidity.


Grafana visualization of temperature and humidity measurements

This is especially evident in the time frame between 22:00 and 23:00, where the two most prominent temperature drops (around 22:15 and 22:30) coincide with a consistent increase in humidity levels. These time intervals correspond to segments of the route passing alongside agricultural fields, which is also confirmed by GPS data.

Presumably, this dynamic may be related to the natural process of moisture evaporation from the soil and vegetation surface, especially under cooling conditions after sunset. As a result, the humid air near the field becomes colder, which is recorded by our sensors as a drop in temperature.

# Conclusion and Future Work

## Summary of main achievements.

Overall, all of the objectives set for this project were successfully achieved within the given timeframe. A test dataset was collected, which allowed zones with the lowest lighting levels to be identified and which also enables real-time data updates — something that can be used to improve urban infrastructure. The dataset also provides opportunities for more active citizen participation in shaping the urban environment through the availability of various feedback tools.

Based on the analysis no significant correlation was made between luminosity and environmental factors such as temperature, humidity and pressure.

All key features of the application, as outlined in the "Objectives" section, were met. However, it should be noted that the interactive map was created in a basic version, without filtering by day and time. Nevertheless, filtering was configured for the dashboard, which also allows data to be tracked over time. Feedback formats were created and implemented to collect data from citizens.
The idea of developing a user profile was set aside, as the primary focus of the project was placed on ensuring data transparency for residents. However, in the authors' view, a sufficiently functional and high-quality proof-of-concept solution was developed, and our hypothesis about the feasibility of using mobile IoT devices for data collection in the context of increasing citizen engagement was confirmed.

All collected data was transmitted correctly, and no hardware-related issues were encountered during testing or data collection. All data is visualized on the web platform. The web platform itself was successfully developed.

Overall, this prototype can be considered a stable and functioning system that may serve as an initial step toward the creation of a finished product, which can be applied in real-world conditions.

## Challenges & Resolutions

The following challenges were encountered during the project execution:

1. Device portability. The original enclosure provided was cumbersome and did not meet the project needs. It was difficult to carry around and did not allow for proper expose of the light sensor which needed not be occluded during the data collection process. The light sensor needed to be exposed to directly face the streetlights. Additionally, it was limited in size and did not allow for proper fitting of all the devices(including power bank) inside. This was solved by designing and printing a 3D enclosure. It was designed such that the LCD screen would be in the outer face and easily readable, a provision

was made to allow for the light sensor to be exposed to directly face the streetlights. Additionally a way to constrain the antenna during movement was also accounted for in the design. All these provisions made the box efficient and fit for the project.

2. Grafana dashboards integration into the web application. Due to access rights, the available Grafana credentials could not to be incorporated into the web app. To overcome this shortcoming the data was fetched and added into the web app using custom dashboards made from appropriate libraries: Chart.js and ApexCharts.js.
3. The first light sensor provided to measure luminosity TSL2561(Digital Light Sensor) wasn't sensitive enough to artificial street lighting at night. It needed to be very close to the light source to give illuminance readings. Therefore, a more sensitive sensor was required. VEML 7700 Ambient Light Senor proved to be a more reliable solution and was able to meet the project needs.
4. The Cayenne LPP being a low powered standard it sometimes ends up rounding off the sensor readings into smaller sizes due to meet space requirements. During testing, it was observed that GPS coordinates were being rounded off into 4 decimal places from 6. This ended up reducing the precision of all GPS readings by up to 10m. This shortcoming meant that the luminosity readings were off and couldn't give the precise locations for the lux readings and also led to clustering of observed values. Unfortunately this could not be sufficiently solved. In the possible future works solving this can be experimented by customizing the Cayenne LPP library and incorporating it in the backend and during development. To offset this shortcoming, a heatmap was created , which allowed for a more visual and intuitive representation of the observations.
5. A different time zone configured for time stamping. The observed values where time stamped with a 2 hour difference to the actual data collection time. This was confusing during the processing and since the project was time specific since it could only be carried out at night. This issue was addressed by using documentation tools such as photographs, from which the sequence of events were reconstructed, and the analysis carried out by correlating location, time, and various lighting and weather parameters.

## Limitations and Possible improvements.

When talking about the limitations of our project, there are several points to mention.

First of all, it does not cover the full spectrum of factors that can influence the sense of safety. Due to our limitations — both financial and in terms of energy consumption — a promising direction for this project would be to expand the number of sensors (for example, a noise sensor to track street activity) connected to our device and to develop a more advanced device that would allow us to assess the factors affecting the sense of safety more deeply.

Another aspect is related to analysis, specifically the way we visualize the data we have. In general, if we had more time to further develop this project and to explore its potential for practical application, it would make sense to create algorithms that allow the data to be

sorted by points of interest and categorized according to streets. Right now, we had to conduct data analysis and lighting analysis for specific streets and street segments manually.
Moreover, the sensor is less reliable with higher lamp posts and depends on the distance to the light source.
Also, more reliable safety metrics like UN SDG indicator 16.1.4 could be calculated if enough citizen feedback is collected. So far, safety evaluation has been performed only by the authors.

In addition, areas with high humidity and lower temperatures are subjectively perceived by one of the project authors as less comfortable and potentially more unsafe. This contradicts the study by Saad et al. (2024), where the opposite is stated, prompting reflection on the cultural context as a factor influencing the perception of safety. Cold and humidity may intensify feelings of anxiety, especially when combined with insufficient lighting and low street activity.

These observations were confirmed by project authors during the route and allow the collected data to be validated as a scientific contribution, opening perspectives for further research by other scholars.

## Recommendations

The following are the recommendations based on this project.

1. The project can be extended to compare measured luminosity to the OpenStreetMap street lighting  layer to create a more robust mapping of illuminance.
2. Realtime monitoring of lighting and environmental factors can be included in the web app to give a realistic feel to the user.
3. The web application can be extended to enrich the user experience for example by providing a real time comment section where users can broadcast alerts to other users, enable issue reporting by uploading pictures among others

# References

## Bibliography

Degbelo, A., Granell, C., Trilles, S., Bhattacharya, D., Casteleyn, S., & Kray, C. (2016). Opening up smart cities: Citizen-centric challenges and opportunities from GIScience. ISPRS International Journal of Geo-Information, 5(2), 16. https://doi.org/10.3390/ijgi5020016

Jacobs, J. (1992). The death and life of great American cities. Vintage Books.

Mora, H., Peral, J., Ferrández, A., Gil, D., & Szymanski, J. (2019). Distributed architectures for intensive urban computing: A case study on smart lighting for sustainable cities. IEEE Access, 7, 1–1. https://doi.org/10.1109/ACCESS.2019.2914613

Saad, R., Portnov, B. A., & Kliger, D. (2024). The feeling of safety by pedestrians at night: An overlooked aspect of climate change? Sustainability, 16(23), 10402. https://doi.org/10.3390/su162310402

Wei, L., Bizjak, G., Svetina, M., & Kobav, M. (2024). Perspectives on urban lighting: Pedestrian safety and comfort preferences. In Proceedings of the BalkanLight 2024 Conference, Istanbul.

Zhu, M., Graham, D. J., Zhang, N., Wang, Z., & Sze, N. N. (2025). Influences of weather on pedestrian safety perception at mid-block crossing: A CAVE-based study. Accident Analysis and Prevention, 215, Article 107988. https://doi.org/10.1016/j.aap.2025.107988

## Web resources and datasets used

Autodesk. (n.d.). *Beginner's guide to 3D printing basics with Fusion 360*. Autodesk. https://www.autodesk.com/products/fusion-360/blog/beginners-guide-3d-printing-basics/

Formlabs. (n.d.). *Types of 3D printing technologies*. https://formlabs.com/3d-printers/?srsltid=AfmBOoosO_6ZXQQ-dJ2W51N5CcXIEKxmrd2c5_CBnCdGSAs4Cp015r3N

HowToMechatronics. (n.d.). *G-code explained: List of most important G-code commands*. https://howtomechatronics.com/tutorials/g-code-explained-list-of-most-important-g-code-commands/

Prusa Research. (n.d.). *Preparing models for 3D printing*. Prusa Knowledge Base. https://help.prusa3d.com/en/category/model-preparation_1777

Technical University of Munich. (n.d.). *Workshop: 3D printing lab*. https://www.hs.mh.tum.de/en/mh/praeventionszentrum/workshop-3d-printing-lab/

Ultimaker. (n.d.-a). *What is FDM 3D printing?* https://ultimaker.com/learn/types-of-3d-printing-technologies/

Ultimaker. (n.d.-b). *Ultimaker Cura*. https://ultimaker.com/software/ultimaker-cura/

All3DP. (n.d.). *STL file explained*. https://all3dp.com/1/stl-file-format-3d-printing/

Statistisches Bundesamt (Destatis). (n.d.). *Proportion of population that feel safe walking alone around the area they live after dark (SDG indicator 16.1.4)*. SDG Indicators for Germany. Retrieved [insert date], from https://sdg-indikatoren.de/en/16-1-4/

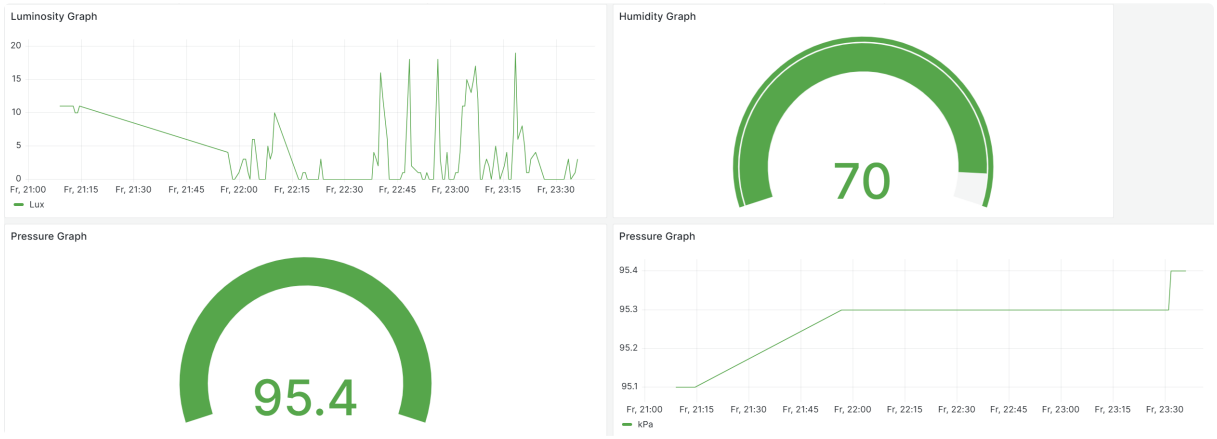
# Appendices

## Raw data samples.

**Sample of raw data of barometric pressure (datastream 1506)**


```
"value": [
    {
      "@iot.selfLink": "https://gi3.gis.lrg.tum.de/frost/v1.1/Observations(231397080)",
      "@iot.id": 231397080,
      "phenomenonTime": "2025-06-13T18:17:05.904Z",
      "resultTime": "2025-06-13T18:17:05.904Z",
      "result": 95.8,
      "Datastream@iot.navigationLink": "https://gi3.gis.lrg.tum.de/frost/v1.1/Observations(231397080)/Datastream",
      "FeatureOfInterest@iot.navigationLink": "https://gi3.gis.lrg.tum.de/frost/v1.1/Observations(231397080)/FeatureOfInterest"
    },
    {
      "@iot.selfLink": "https://gi3.gis.lrg.tum.de/frost/v1.1/Observations(231397082)",
      "@iot.id": 231397082,
      "phenomenonTime": "2025-06-13T18:17:05.904Z",
      "resultTime": "2025-06-13T18:17:05.904Z",
      "result": 95.8,
      "Datastream@iot.navigationLink": "https://gi3.gis.lrg.tum.de/frost/v1.1/Observations(231397082)/Datastream",
      "FeatureOfInterest@iot.navigationLink": "https://gi3.gis.lrg.tum.de/frost/v1.1/Observations(231397082)/FeatureOfInterest"
    },
    {
      "@iot.selfLink": "https://gi3.gis.lrg.tum.de/frost/v1.1/Observations(231397191)",
      "@iot.id": 231397191,
      "phenomenonTime": "2025-06-13T18:18:13.88Z",
      "resultTime": "2025-06-13T18:18:13.88Z",
      "result": 95.8,
      "Datastream@iot.navigationLink": "https://gi3.gis.lrg.tum.de/frost/v1.1/Observations(231397191)/Datastream",
      "FeatureOfInterest@iot.navigationLink": "https://gi3.gis.lrg.tum.de/frost/v1.1/Observations(231397191)/FeatureOfInterest"
    },
}]
```

# Grafana dashboards.




Grafana dashboard